L-BFGS-B

3.0

Generated by Doxygen 1.9.1

1 L-BFGS-B

Software for Large-scale Bound-constrained Optimization

L-BFGS-B is a limited-memory quasi-Newton code for bound-constrained optimization, i.e., for problems where the only constraints are of the form `l <= x <= u`. It is intended for problems in which information on the Hessian matrix is difficult to obtain, or for large dense problems. L-BFGS-B can also be used for unconstrained problems, and in this case performs similarly to its predecessor, algorithm L-BFGS (Harwell routine VA15). The algorithm is implemented in Fortran 77.

Authors

- Ciyou Zhu

- Richard Byrd
- Jorge Nocedal
- Jose Luis Morales

Related Publications

- R. H. Byrd, P. Lu, J. Nocedal and C. Zhu. A Limited Memory Algorithm for Bound Constrained Optimization (1995), SIAM Journal on Scientific and Statistical Computing, Vol. 16, Num. 5, pp. 1190-1208
- C. Zhu, R. H. Byrd and J. Nocedal. L-BFGS-B: Algorithm 778: L-BFGS-B, FORTRAN routines for large scale bound constrained optimization (1997), ACM Transactions on Mathematical Software, Vol. 23, Num. 4, pp. 550-560
- J.L. Morales and J. Nocedal. L-BFGS-B: Remark on Algorithm 778: L-BFGS-B, FORTRAN routines for large scale bound constrained optimization (2011), ACM Transactions on Mathematical Software, Vol. 38, Num. 1
- R. H. Byrd, J. Nocedal and R. B. Schnabel. Representations of quasi-Newton matrices and their use in limited memory methods (1994), Mathematical Programming, Vol. 63, pp. 129-156

Note that the subspace minimization in the LBFGSpp implementation is an exact minimization subject to the bounds, based on the BOXCQP algorithm:

- C. Voglis and I. E. Lagaris, BOXCQP: An Algorithm for Bound Constrained Convex Quadratic Problems (2004), 1st International Conference "From Scientific Computing to Computational Engineering", Athens, Greece

For an eagle-eye overview of L-BFGS-B and the genealogy BFGS->L-BFGS->L-BFGS-B, see Henao's Master's thesis.

Related Software

- wilmerhenao/L-BFGS-B-NS: An L-BFGS-B-NS Optimizer for Non-Smooth Functions
- pcarbo/lbfgsb-matlab: A MATLAB interface for L-BFGS-B
- bgranzow/L-BFGS-B: A pure Matlab implementation of L-BFGS-B (LBFGSB)
- constantino-garcia/lbfgsb_cpp_wrapper: A simple C++ wrapper around the original Fortran L-BGSG-B routine
- yixuan/LBFGSpp: A header-only C++ library for L-BFGS and L-BFGS-B algorithms
- chokkan/liblbfgs: libLBFGS: a library of Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS)
- mkobos/lbfgsb_wrapper: Java wrapper for the Fortran L-BFGS-B algorithm
- yuhonglin/Lbfgsb.jl: A Julia wrapper of the l-bfgs-b fortran library
- Gnimuc/LBFGSB.jl: Julia wrapper for L-BFGS-B Nonlinear Optimization Code
- afbarnard/go-lbfgsb: L-BFGS-B optimization for Go, C, Fortran 2003
- nepluno/lbfgsb-gpu: An open source library for the GPU-implementation of L-BFGS-B algorithm

- Chris00/L-BFGS-ocaml: OCaml bindings for L-BFGS
- dwicke/L-BFGS-B-Lua: L-BFGS-B lua wrapper around a L-BFGS-B C implementation
- avieira/python_lbfgsb: Pure Python-based L-BFGS-B implementation
- ybyygu/rust-lbfgsb: Ergonomic bindings to L-BFGS-B code for Rust
- rforge/lbfgsb3c: Limited Memory BFGS Minimizer with Bounds on Parameters with optim() 'C' Interface for R
- florafauna/optimParallel-python: A parallel version of 'scipy.optimize.minimize(method='L-↩ BFGS-B')`

Notes on this repository

I (J. Schilling) took the freedom to

- put the L-BFGS-B code obtained from the original website up in this repository,
- divide the subroutines and functions into separate files,
- convert parts of the documentation into a format understandable to doxygen and
- replace the included BLAS/LINPACK routines with calls to user-provided BLAS/LAPACK routines. To be precise, the calls to LINPACK's `dtrsl` were replace with calls to LAPACK's `dtrsm` and the calls to LINPACK's `dpofa` were replaces with calls to LAPACK's `dpotrf`.

Porting to another language

A language-neutral specification of the algorithm – sufficient to implement L-BFGS-B in any language with a BLAS/LAPACK binding, without reading the Fortran source – lives in `docs/spec/`. Start with `docs/spec/README.md`.

The pack includes:

- 9 foundation documents (algorithm overview, glossary, abstract callback-based API, logical state model, numerical conventions, deviations from the published papers, conformance criteria, F77 -> other-language gotchas, and an optional appendix on the F77 reverse-communication interface for ABI compatibility).
- One per-subroutine spec for each in-scope numerical routine.
- ∼110 JSON test vectors mirroring the F77 unit-test cases.
- A Python+NumPy reference implementation (`docs/spec/reference_impl/`).
- A conformance test runner (`docs/spec/runner/conformance.py`) with strict (bit-exact) and tolerance (per-subroutine numerical) modes.

Building

A CMake setup is provided for L-BFGS-B in this repository. External modules for `BLAS` and `LAPACK` have to be installed on your system. Then, building the shared library `liblbfgsb.so` and the examples `driver*.f` and `driver*.f90` works as follows:

```
> mkdir build
> cd build
> cmake ..
> make -j
```

The resulting shared library and the driver executables can be found in the `build` directory.

Concluding remarks

The current release is version 3.0. The distribution file was last changed on 02/08/11.

This work was in no way intending to infringe any copyrights or take credit for others' work. Feel free to contact me at any time in case you noticed something against the rules. Above documentation is obtained from the archived version of the `original manual`.

A PDF version of the documentation is available here: `L-BFGS-B.pdf`.

2 Algorithm overview

L-BFGS-B minimizes a smooth function `f : R^n -> R` subject to box bounds `l <= x <= u`, using a limited-memory quasi-Newton approximation to the Hessian and a projected gradient framework for the bounds. Per iteration:

1. **Identify the active bound set.** Variables that the search direction would push into a bound are fixed at that bound; the rest are *free*.
2. **Compute the generalized Cauchy point** `x_c`: the minimizer of a quadratic model of `f` along the projected gradient direction. This determines which bounds become active in this iteration.
3. **Subspace minimization.** Approximately solve the box-constrained quadratic subproblem on the free variables, using the L-BFGS Hessian approximation. This yields a search direction `d`.
4. **Line search** along `d` (More-Thuente Wolfe-condition line search), projected onto the box. The accepted step `alpha*d` produces the next iterate `x_{k+1}`.
5. **Update the L-BFGS history**: append `(s_k, y_k) = (x_{k+1} - x_k, g_{k+1} - g_k)` to the limited-memory pair list, dropping the oldest pair if the memory bound `m` is reached.
6. **Test convergence** on `||projected gradient||_Inf < pgtol` and on relative function decrease `(f_k - f_{k+1}) / max(|f_k|, |f_{k+1}|, 1) < factr * epsmch`.

The algorithm runs until convergence, an iteration/evaluation limit is hit, or the line search fails (which can indicate the function is unbounded below, the gradient is inconsistent with the function, or the problem is too ill-conditioned).

The "limited memory" part: instead of storing the `n*n` Hessian approximation, we store the last `m` pairs `(s_i, y_i)`, with `m` typically 5-17. The compact representation (Byrd/Nocedal/Schnabel 1994) lets us compute Hessian-vector products in `O(mn)` time without ever materializing the dense Hessian.

Paper map

L-BFGS-B is described across three papers. `**code.pdf` is the authoritative reference** – it documents the algorithm as actually implemented in this codebase. The others give derivations or fixes.

`algorithm.pdf` – Byrd/Lu/Nocedal/Zhu, "A Limited Memory Algorithm for Bound Constrained Optimization" (SIAM J. Sci. Comput. 1995)

The original algorithm derivation. Read this for:

- The mathematical justification for the active-set strategy.
- The generalized Cauchy point definition (eq 4.6).
- The compact L-BFGS representation (also covered in Byrd/Nocedal/Schnabel 1994).
- The original subspace minimization scheme.

Caveat: the implemented code follows the variant in `code.pdf`, not this paper. The paper states this explicitly: ∗"the algorithm we present below differs in a few important details from the one originally described."∗ See `05_deviations.md`.

`code.pdf` – Zhu/Byrd/Nocedal, "L-BFGS-B: Algorithm 778" (ACM TOMS 1997)

The algorithm as implemented. **This is the reference for the spec.** Covers:

- The iteration as actually coded, including the line search wrapper (`lnsrlb`), the curvature test, and the refresh logic.
- The reverse-comm interface (`setulb`).
- Termination tests (`factr`, `pgtol`).
- Workspace-array layout (relevant only for the F77 implementation; ports do not need to follow this).

`acm-remark.pdf` – Morales/Nocedal, "Remark on Algorithm 778" (ACM TOMS 2011)

A correction to the subspace minimization in `subsm` (the "Cauchy-direction" variant). The original 1997 code had a bug in how it handled bounds during the subspace solve; this paper documents the fix. The fix is included in `src/subsm.f`.

Reading order if you are porting

1. Read `algorithm.pdf` sec.1-sec.5 for intuition (1-2 hours).
2. Skim `code.pdf` sec.1-sec.3 for the coded algorithm and the termination tests (30 minutes).
3. Skim `acm-remark.pdf` (15 minutes) – only matters for `subsm`.
4. Then read this pack starting from `01_glossary.md`.

You do not need to read `code.pdf` sec.4 (Fortran implementation details) unless you want to mimic the F77 reverse-comm interface, in which case also read this pack's `08_legacy_reverse_comm.md`.

Algorithm complexity per iteration

For dimension `n` and memory `m` (typically `n >> m`):

Step	Work
Identify active set (active/cauchy)	O(n + m^2)
Cauchy point breakpoint sort (hpsolb)	O(n log n) worst case
Subspace minimization (subsm)	O(m^2 n)
Line search (lnsrlb)	O(n) per evaluation, plus user f/g cost
L-BFGS update (matupd, formt)	O(m^2 + mn)

Memory: O(mn + m^2). The F77 wa workspace is sized 2mn + 5n + 11m^2 + 8m double-precision values; the iwa integer workspace is 3n integers. Ports may use whatever data structures fit their language as long as the per-iteration math matches.

3 Glossary: notation across paper, code, and spec

This document maps the notation used in the three reference papers (algorithm.pdf, code.pdf, acm-remark.pdf) to the F77 identifiers in src/ and the spec notation used throughout this pack.

When papers disagree on notation, the **Spec** column gives the canonical name used here; per-subroutine specs in subroutines/ use this notation.

Scalars and counters

Spec	Paper	F77	Description
n	n	n	Number of variables (problem dimension).
m	m	m	Maximum number of L-BFGS pairs to retain. Typical 5-17.
col	k (alg) / m_k (code)	col	Current number of stored L-BFGS pairs (0 <= col <= m). Increases each iteration until reaching m, then stays at m.
iter	k	iter	Iteration counter (0-based; iter=0 is the initial point).

Spec	Paper	F77	Description
theta	theta	theta	Hessian scaling parameter; 'theta = (y_k' y_k) / (s_k' y_k)after each successful update. Initialized to 1. \ilinebr </td> </tr> <tr class="markdown↩ TableRowEven"> <td class="markdown↩ TableBodyNone">f\ilinebr </td> <td class="markdownTableBodyNone">f(x)\ilinebr </td> <td class="markdownTableBody↩ None">f\ilinebr </td> <td class="markdown↩ TableBodyNone"> Function value at current iterate. \ilinebr </td> </tr> <tr class="markdownTableRowOdd"> <td class="markdownTableBodyNone">factr\ilinebr </td> <td class="markdownTable↩ BodyNone">factr\ilinebr </td> <td class="markdownTableBodyNone">factr\ilinebr </td> <td class="markdownTableBodyNone"> Function-decrease tolerance (relative). Convergence when(f_old - f_new) / max(|f_old|, |f↩ _new|, 1) <= factr * eps. Typical10^7(low accuracy),10^1(machine accuracy). \ilinebr </td> </tr></table> |pgtol|pgtol|pgtol| Projected-gradient tolerance. Convergence when||g_proj||_Inf <= pgtol. | |eps/epsmch|eps_M|epsmch| Machine epsilon (∼=2.22 * 10^{-16}` for IEEE-754 double). |

Vectors (current state)

Spec	Paper	F77	Shape	Description
x	x_k	x	n	Current iterate.
g	g_k = gradf(x↩ _k)	g	n	Gradient at current iterate.
g_proj	|P[x - g, l, u] - x|_Inf	(computed in projgr)	scalar	Infinity norm of projected gradient.
s	s_k = x_↩ {k+1} - x_k	(incremental)	n	Step from previous iterate.
y	y_k = g_↩ {k+1} - g_k	(incremental)	n	Gradient difference.
d	d_k	d	n	Search direction (subspace minimizer).
xc	x^c	z	n	Generalized Cauchy point.
xp	x_↩ {k-1}	xp	n	Previous iterate (kept for refresh logic).
r	(varies)	r	n	Reduced gradient at the Cauchy

Spec	Paper	F77	Description

4 Algorithm interface

The canonical L-BFGS-B interface is a callback-based function. The caller passes the initial point, the bounds, the algorithm parameters, and two closures that evaluate the objective function and its gradient on demand. The optimizer drives the callbacks internally and returns the final iterate.

This is the natural interface for languages with first-class function values. Ports are free to expose their own user-facing API on top of this – generators, async iterators, builder patterns, observer callbacks, or even the F77 reverse-comm protocol described in `08_legacy_reverse_comm.md` – but the spec describes the algorithm in callback terms.

Signature

In language-neutral pseudocode:

```
result minimize(
    n: positive integer,                    # dimension
    x0: real vector of length n,            # initial point
    l: real vector of length n,             # lower bounds
    u: real vector of length n,             # upper bounds
    nbd: integer vector of length n,        # bound type per variable
    m: positive integer,                    # max L-BFGS pairs (typical 5-17)
    f_eval: function(x: real vector) -> real,
    g_eval: function(x: real vector) -> real vector of length n,
    factr: nonneg real = 1e7,               # function-decrease tolerance
    pgtol: nonneg real = 1e-5,              # projected-gradient tolerance
    max_iter: positive integer = none,      # iteration cap (none = unlimited)
    max_fg: positive integer = none,        # f/g evaluation cap
    iprint: integer = -1,                   # diagnostic verbosity
)
```

Returns:

```
result {
    x: real vector of length n,         # final iterate
    f: real,                            # f(x) at final iterate
    g: real vector of length n,         # gradient at final iterate
    info: integer,                      # termination code (see below)
    n_iter: integer,                    # iterations performed
    n_fg: integer,                      # f/g callback invocations
    final_projg: real,                  # final ||g_proj||_inf
    message: string,                    # human-readable termination reason
}
```

Inputs

`n` – dimension

Positive integer. Must be `>= 1`.

`x0` – initial point

Real vector of length `n`. May be infeasible: the optimizer will project onto the box `l <= x <= u` before iterating; the caller is notified via `info` if projection occurred.

`l`, `u` – bounds

Real vectors of length `n`. Entries are **only consulted** where `nbd[i]` indicates the corresponding bound is active (see below). Entries with `nbd[i] = 0` may be any value (commonly `0`).

`nbd` – bound types

Integer vector of length `n`. Per variable:

nbd[i]	Meaning
0	Variable is unbounded. l[i], u[i] ignored.
1	Lower bound active: x[i] >= l[i].
2	Both bounds active: l[i] <= x[i] <= u[i]. Allowed l[i] = u[i] (variable fixed).
3	Upper bound active: x[i] <= u[i].

Validation: nbd[i] in {0,1,2,3} is required; if nbd[i] = 2 then l[i] <= u[i] is required. Violations cause an immediate return with info set to an error code (see errclb.md).

m – memory parameter

Positive integer. Number of L-BFGS pairs (s_i, y_i) retained. Typical values: 5 (default), up to 17. Larger m improves the Hessian approximation at the cost of O(mn) extra storage and O(m^2n) per-iteration work.

f_eval, g_eval – callbacks

Pure functions called by the optimizer. Contract:

- **Pure**: a port may rely on f_eval(x) = f_eval(x) for the same x (the optimizer may call them with the same x more than once, although it tries to avoid this).
- **Inputs**: f_eval and g_eval receive the current trial iterate x. The optimizer guarantees l <= x <= u (the trial point is always feasible after the initial projection).
- **Outputs**: f_eval returns a finite real (NaN / Inf cause abnormal termination); g_eval returns a real vector of length n.
- **Cost**: the optimizer minimizes calls but cannot avoid them – for expensive objectives, the user controls cost via max_fg and factr/pgtol tolerances.

A port may merge f_eval and g_eval into a single callback returning both (f, g); this is more efficient when the gradient is computed as a byproduct of the function evaluation. Both styles are valid; this spec describes them as separate callbacks for clarity.

factr – relative function-decrease tolerance

Convergence triggers when
```
(f_old - f_new) / max(|f_old|, |f_new|, 1) <= factr * eps
```

where eps is the machine epsilon (≈ 2.22e-16). Recommended values:

- factr = 1e12: low accuracy (≈6 digits).
- factr = 1e7: medium accuracy (≈10 digits, default).
- factr = 1e1: machine accuracy (≈15 digits).

Setting factr = 0 disables the test (only pgtol and limits remain).

pgtol – projected-gradient tolerance

Convergence triggers when ||g_proj||_Inf <= pgtol, where g_proj is the projected gradient (see subroutines/projgr.md for the exact definition).

Setting pgtol = 0 disables the test.

max_iter, max_fg – limits

Optional caps. If reached, the optimizer terminates with info = INFO_LIMIT (see below). The F77 default in the drivers is max_iter = 99 and unlimited max_fg; ports may choose their own defaults.

iprint – diagnostic verbosity

Value	Behavior
< 0	Silent (default for libraries).
0	Print only at start and end.
1 to 98	Print every iprint-th iteration; final summary.
99	Print details at every iteration.
100	Plus print Cauchy point variables and active-set changes.
> 100	Plus print working vectors at each iteration.

Diagnostic format is **port-defined**. The F77 implementation writes to unit 6 and an iterate.dat file; ports may use any logging mechanism. Conformance tests do not check diagnostic output.

Termination codes (info)

| Code | Symbolic name | Meaning | |---|---|---| | 0 | INFO_CONVERGED_PGTOL | ||g_proj||_↩ Inf <= pgtol. | | 1 | INFO_CONVERGED_FACTR | Relative function decrease below factr * eps. | | 2 | INFO_LIMIT_ITER | Reached max_iter without converging. | | 3 | INFO_LIMIT_FG | Reached max_fg without converging. | | 4 | INFO_ABNORMAL_LNSRLB | Line search failed (function may be unbounded below, or gradient is inconsistent with f). | | 5 | INFO_USER_STOP | User requested early termination. | | <= -1 | INFO_INPUT_ERROR_* | Input validation failed. See errclb.md for the specific code per validation. |

Ports may use enums, exceptions, or sentinel values to expose this information; the codes above are the spec convention.

The message string in the result is a human-readable description suitable for logging. The F77 implementation uses the task string; ports may produce any text but should match the meaning of the corresponding info code.

Behavioral guarantees

- **Initial projection**: if any x0[i] is outside its bounds, the optimizer projects to the nearest feasible point before any callback is invoked. The callbacks always receive feasible points.
- **Monotonic decrease (within tolerance)**: each iteration either reduces f or terminates. The line search (More-Thuente Wolfe) enforces sufficient decrease and curvature conditions; rejected steps trigger refresh of the L-BFGS history rather than accepting an ascent step.
- **Bounded callback count**: the algorithm performs at most max_iter outer iterations and at most max_fg total callback evaluations.
- **Determinism**: given identical inputs, identical callbacks, and a reference BLAS/LAPACK pin (see 04_↩ numerics.md), two runs produce bit-identical outputs. The algorithm has no randomness.

Behavior on invalid input

The optimizer validates inputs before the first callback. On validation failure it returns immediately with `info <= -1` and **no callbacks have been invoked**. See `errclb.md` for the full list of error cases and codes.

NaN / Inf in the user-supplied `x0`, `l`, or `u` is allowed only where the bound is inactive (e.g., `u[i]` may be `+Inf` if `nbd[i] in {0, 1}`). NaN / Inf returned from `f_eval` or `g_eval` causes abnormal termination with `info = INFO_ABNORMAL_LNSRLB`.

Optional: stopping callback

Some ports may want to allow the user to interrupt the optimization between iterations (e.g., timeout, external signal). The F77 `setulb` exposes this via 'task = 'STOP'` mid-iteration.

For a callback-based port, the recommended idiom is an optional `should_stop: function(state) -> boolean` callback invoked at each iteration boundary; if it returns `true`, the optimizer terminates with `info = INFO_USER_STOP`. The state passed to `should_stop` may include `iter`, `n_fg`, current `f`, current `||g_proj||_Inf`, and elapsed wall time – port's choice.

This is **not part of the algorithm spec** (the algorithm itself does nothing different); it is an interface convenience that ports may or may not provide.

Mapping to F77 `setulb`

For readers comparing to the F77 source:

Spec parameter	F77 equivalent
`n`, `m`, `x0`, `l`, `u`, `nbd`	Same names.
`factr`, `pgtol`, `iprint`	Same names.
`f_eval`, `g_eval`	Caller computes `f`, `g` between `setulb` calls when 'task = 'FG'. \ilinebr </td> </tr> <tr class="markdownTableRowEven"> <td class="markdownTableBodyNone">max_iter, max_fg\ilinebr </td> <td class="markdownTableBodyNone"> F77 has no caps in setulb itself; the drivers loop and break on task != 'FG' && task != 'NEW_X'. \ilinebr </td> </tr> <tr class="markdownTableRowOdd"> <td class="markdownTableBodyNone"> Result x, f, g, info\ilinebr </td> <td class="markdownTableBodyNone"> F77 leaves these in-place in the user's arrays. \ilinebr </td> </tr> <tr class="markdownTableRowEven"> <td class="markdownTableBodyNone"> Result n_iter, n_fg\ilinebr </td> <td class="markdownTableBodyNone"> F77 stores in isave(30), isave(34). \ilinebr </td> </tr> <tr class="markdownTableRowOdd"> <td class="markdownTableBodyNone"> Result message\ilinebr </td> <td class="markdownTableBodyNone"> F77 returns in task`.

The full F77 protocol is in `08_legacy_reverse_comm.md`.

5 Logical state model

This document describes the algorithm's state in terms of logical objects: what L-BFGS-B holds in memory, what changes per iteration, and how state is initialized and refreshed.

It does **not** describe the F77 `wa` / `iwa` workspace packing – that is an implementation choice of the F77 source. Ports may use any storage layout.

Overview

The state divides into three groups:

1. **Persistent state** – survives across iterations. Updated each successful iteration.
2. **Transient state** – built and discarded within a single iteration.
3. **Reverse-comm state** ∗(legacy only)∗ – only meaningful for ports that mimic the F77 `setulb` reverse-comm interface. Encodes how to resume mid-iteration. See `08_legacy_reverse_comm.md`.

A callback-based port (the canonical interface; see `02_api.md`) needs groups 1 and 2 only. The "state" between callbacks is the natural local-variable state of the running optimization function – no packing required.

Persistent state

Held across iteration boundaries. After an iteration completes, this is what the optimizer needs to start the next one.

Iterate

Object	Type	Description
`x`	real vector, length `n`	Current iterate. Always feasible (`l <= x <= u`).
`f`	real	`f(x)` at current iterate.
`g`	real vector, length `n`	`gradf(x)` at current iterate.
`xp`	real vector, length `n`	Previous iterate (used for refresh decisions).

Counters

Object	Type	Description
`iter`	integer >= 0	Iteration counter; `iter = 0` is the initial point.
`n_fg`	integer >= 0	Total callback invocations so far.

L-BFGS history

The history is a **logical pair list**, ordered from oldest to newest: `[(s_1, y_1), (s_2, y_2), ..., (s_col, y_col)]`. After each successful iteration with a sufficiently positive curvature, a new pair is appended; if `col = m` is already at the limit, the oldest pair is dropped first.

Object	Type	Description
col	integer, 0 <= col <= m	Current pair count.
S	real matrix, n * col	Columns are s_i = x_{k_i+1} - x_{k_i} in insertion order.
Y	real matrix, n * col	Columns are y_i = g_{k_i+1} - g_{k_i} in insertion order.
theta	real, > 0	Hessian scaling (‘y_k' y_k / s_k' y_kafter each update; initialized to1`).

Ports may store S and Y as column-major matrices, row-major matrices, lists of vectors, or any other structure. The F77 source uses column-major n * m arrays with head/itail indices for cyclic storage when col = m; this is one valid choice but not required.

Cached quantities for the compact representation

Maintained alongside S, Y to avoid recomputing O(col^2 * n) work per iteration. **All three are derived deterministically from S, Y, and theta.** A port may recompute on demand instead of caching; the only consequence is more O(col^2 n) work per iteration. If cached, they must be updated when (S, Y, theta) changes.

Object	Type	Defined as
sy	real matrix, col * col	‘sy[i,j] = s_i' * y_j. The diagonal isD = diag(s↩ _i' y_i). The strict lower triangle isL_{i,j} = s_i' y_jfori > j. The strict upper triangle is unused (in the F77 packing). \ilinebr </td> </tr> <tr class="markdownTable↩ RowEven"> <td class="markdownTableBody↩ None">ss\ilinebr </td> <td class="markdown↩ TableBodyNone"> real matrix,col * col\ilinebr </td> <td class="markdownTableBodyNone">ss[i,j] = s_i' * s_j. Symmetric; the F77 source stores only the upper triangle. \ilinebr </td> </tr> <tr class="markdownTableRowOdd"> <td class="markdownTableBodyNone">T\ilinebr </td> <td class="markdownTableBodyNone"> real matrix,col * col, upper-triangular \ilinebr </td> <td class="markdownTableBodyNone"> Cholesky factor:T' T = theta * S'S + L * D^{-1} * L'(the Cholesky factor of the middle matrixM_inv's lower-right block). Computed byformtfromsy,ss,theta`.

Bound state

Object	Type	Description
iwhere	integer vector, length n	Per-variable bound status: -1 (unbounded), 0 (free, has bounds), 1 (at lower), 2 (at upper), 3 (fixed). See 01_glossary.md. Initialized by active; refined by cauchy.
prjctd	boolean	true if the initial x0 was infeasible and projected. Set once by active at startup.
cnstnd	boolean	true if any variable has at least one bound. Set once at startup.
boxed	boolean	true if every variable has both lower and upper bounds. Set once at startup.

Refresh tracking

Object	Type	Description
updatd	boolean	true if the L-BFGS history was updated in the *previous* iteration. Used by cauchy and subsm to decide whether T and K factorizations need recomputation.
head	integer	F77 implementation detail: index of oldest column in the cyclic S/Y buffer. Ports using non-cyclic storage do not need this.

Transient state (per iteration)

Built up during one iteration and discarded at the iteration boundary. A callback-based port can hold these as local variables in the iteration loop.

Cauchy step

Object	Type	Description
xc	real vector, length n	Generalized Cauchy point. Computed by cauchy.
c	real vector, length 2 col	'c = W' (xc - x); built bycauchy, used bysubsm`.

Active set after Cauchy

Object	Type	Description
nfree	integer	Number of currently free variables.
index	integer vector, length n	Permutation: index[1..nfree] are the free-variable indices; index[nfree+1..n] are the active (bound) ones.
indx2	integer vector, length n	Indices of variables that *changed* status this iteration↩ : indx2[1..nenter] entered the free set; indx2[ileave..n] leaved.
nenter	integer	Number of variables that became free this iteration.
ileave	integer <= n+1	Index in indx2 where leaving variables start (n+1 means none left).

Subspace step

Object	Type	Description
r	real vector, length n	Reduced gradient at the Cauchy point on the free variables.
d	real vector, length n	Search direction for the line search.

Line search

Object	Type	Description
stp	real, > 0	Step length along d.
xnew	real vector, length n	Trial point x + stp * d.
f_new, g_new	real, real vector	Trial function value and gradient.
(line-search internal state)	–	Bracket interval [stx, sty], plus best/end-point function and slope values. See dcsrch.md for the full set.

Diagnostics and counters (per iteration)

| Object | Type | Description | |-------|-----|-----------| | `g_proj_inf` | real | ||g_proj||_Inf at the start of the iteration. Used for the convergence test. | | `f_old` | real | `f` at the start of the iteration; used by the relative-decrease convergence test. | | `nfgv` | integer | Callbacks consumed within this iteration's line search. | | `nint` | integer | Internal step-count counter (cumulative across iterations). |

Initialization

Before the first iteration:

1. **Validate inputs** (`errclb`). Failure -> return with `info < 0`.
2. **Project x0** to the feasible set if needed (`active`); set `iwhere`, `prjctd`, `cnstnd`, `boxed`.
3. **Allocate** `S`, `Y` empty (`col = 0`); set `theta = 1`.
4. **Set counters**: `iter = 0`, `n_fg = 0`, `updatd = false`.
5. **First evaluation**: invoke `f_eval(x0)` and `g_eval(x0)` to get `f` and `g`. Increment `n_fg`.
6. **Initial convergence check**: compute `g_proj_inf`; if it is `<= pgtol`, return with `info = INFO_↩ CONVERGED_PGTOL` immediately.

The legacy reverse-comm version splits steps 1-4 into the `START` task and yields after step 5 with ‘task = 'FG'`; ports using callbacks collapse this into straight code.

Per-iteration update

After computing the next iterate `x_new` with function `f_new` and gradient `g_new`:

1. **Check curvature**: compute `s = x_new - x` and `y = g_new - g`. If ‘s’ y / |s' s| > eps * ||y||^2 (a refresh threshold; seemainlb.md` for the exact formula), accept the pair.
2. **If accepted**: append `(s, y)` to `S`, `Y`. If `col = m` already, drop the oldest column first. Update ‘theta = (y' y) / (s' y). Update cachedsy, ss. RecomputeT = formt(sy, ss, theta). Set updatd = true.
3. **If rejected** (curvature too small orformtCholesky failed): discard the proposed pair; setupdatd = false. Thecauchyand subsm` calls in the next iteration will reuse the existing factorizations.
4. **Set** `xp = x`, `x = x_new`, `f = f_new`, `g = g_new`. Increment `iter`.

Refresh / restart

In adverse cases (e.g., the line search fails, or repeatedly rejects steps), the algorithm "refreshes" by discarding all L-BFGS history and starting from the current point with `col = 0`, `theta = 1`. This effectively resets the Hessian approximation to the identity. See `mainlb.md` and `lnsrlb.md` for the exact triggers.

A refresh is a complete reset of the L-BFGS history; persistent state *outside* the history (`x`, `f`, `g`, `iwhere`, counters) survives.

State at termination

When the optimizer returns, the persistent state contains:

- The final iterate `x` and its `f`, `g`.
- Counters `iter`, `n_fg`.
- Termination reason in `info` and `message` (see `02_api.md`).

Transient state and L-BFGS history are no longer needed and may be discarded. Ports that wrap the optimizer in an object are free to retain history if useful (e.g., for warm-starting a follow-up optimization), but the spec does not require this.

Cardinality summary

For dimension `n` and memory `m`:

Storage class	Approximate size (doubles)
Persistent – iterate (`x`, `g`, `xp`)	`3n`
Persistent – L-BFGS history (`S`, `Y`)	`2 * m * n`
Persistent – cached (`sy`, `ss`, `T`)	`3 * m^2`
Transient – Cauchy/subsm (`xc`, `c`, `r`, `d`, `K`, `K_chol`)	∼`5n + 8m^2`

Total typical: `5n + 2mn + 11m^2` doubles, plus `O(n)` integer state. This matches the F77 `wa` (`2mn + 5n + 11m^2 + 8m`) and `iwa` (`3n`) workspace sizes; the extra `8m` is small scratch in `cauchy`/`bmv` that ports can also hold as locals.

6 Numerical considerations

This document covers floating-point conventions, tolerances, and the BLAS/LAPACK reference-build policy that ports must follow to achieve bit-for-bit conformance.

The full magic-constants table (every numerical literal in the F77 source with its origin and whether ports may vary it) appears at the end and is filled in during the deviations audit (see Phase D in the plan).

Precision

L-BFGS-B operates in **IEEE-754 double precision throughout**. All real values are 64-bit binary doubles with normal/subnormal handling, NaN propagation, and rounding mode = round-to-nearest-even (the IEEE default).

Single-precision is **not supported**. The algorithm's ill-conditioning behavior near convergence requires the headroom of double precision; ports that wrap the algorithm in a single-precision interface internally upcast to double for the algorithm itself.

Extended precision (`long double`, x87 80-bit) **must not be used**. Some legacy compilers default to extended precision in registers; this breaks bit-for-bit reproducibility against the F77 reference. Ports on x86 must explicitly enforce `-msse2` (or equivalent SSE2 / FPU control word setting) to keep all arithmetic in 64-bit IEEE-754.

Machine epsilon

The algorithm uses machine epsilon (`eps`) in tolerance computations.

Spec	F77	Value
eps	epsmch	~= 2.220446049250313e-16 (= 2^{-52})

The F77 source recomputes epsmch at startup in mainlb via:

```
epsmch = 1
do while (1 + epsmch /= 1)
   epsmch = epsmch / 2
end do
epsmch = epsmch * 2
```

Ports may either:

- Use the same iterative computation (guaranteed to produce the same result on IEEE-754 hardware), or
- Use the language's built-in DBL_EPSILON / numpy.finfo(float).eps / equivalent, which on conforming IEEE-754 platforms equals 2.220446049250313e-16.

Both choices produce identical tolerance arithmetic.

Convergence tolerances

Function-decrease tolerance: factr

The F77 user passes factr (default 1e7) and the algorithm tests:

```
(f_old - f_new) / max(|f_old|, |f_new|, 1.0) <= factr * eps
```

Equivalent to: relative function decrease <= factr * eps. With factr = 1e7, this is roughly 2.2e-9 – about 9 digits of accuracy. With factr = 1e1, it is roughly 2.2e-15 – close to machine precision.

Setting factr = 0 disables this convergence test.

Projected-gradient tolerance: pgtol

```
||g_proj||_Inf <= pgtol
```

where g_proj is the projected gradient defined in subroutines/projgr.md. Default pgtol = 1e-5. Setting pgtol = 0 disables this test.

Both tests can fire

The algorithm tests both factr and pgtol after each iteration. Whichever fires first determines the info code (see 02_api.md).

Line-search tolerances

The More-Thuente line search (dcsrch) uses three internal tolerances plus step-length bounds.

Symbol	F77	Value	Meaning
ftol	ftol	1.0e-3	Sufficient-decrease (Armijo) coefficient.
gtol	gtol	0.9	Curvature condition coefficient.
xtol	xtol	0.1	Bracket-width relative tolerance.
stpmin	stepmin	0.0	Minimum allowed step.
stpmax	stepmx	computed	Max step (limited by box geometry).

ftol = 1e-3 is a documented deviation from the published algorithm, which prescribes alpha = 1e-4 in More-Thuente 1994. The L-BFGS-B implementation uses 1e-3 and has been validated against the test suite at this value. See lnsrlb.md for the comment in src/lnsrlb.f explaining this. **Ports must use ftol = 1e-3** (not 1e-4) to be conformant. If we ever change this value, conformance test bounds will need to be regenerated.

Numerical safeguards

Division guards

The algorithm performs several divisions where a denominator near zero would be catastrophic. Each is guarded:

Site	Guard	Behavior on guard fire
cauchy: breakpoint computation (u - x) / d	implicit (skip if d = 0)	Variable treated as having no breakpoint in this direction.
cauchy: dt = -fp / fpp	check fpp > eps (else use earlier breakpoint)	Algorithm exits the breakpoint loop.
formt: Cholesky pivot T[i,i]^2 > 0	LAPACK dpotrf returns info > 0	Caller (mainlb) signals refresh; L-BFGS history discarded.
bmv: 1 / sqrt(sy[i,i])	precondition: sy[i,i] > 0 enforced by matupd	If precondition violated, behavior undefined.

| matupd: s'y curvature test | s'y > eps * ||y||^2 (refresh threshold) | New pair rejected; updatd = false. |

Overflow / underflow

The algorithm does not explicitly guard against IEEE-754 overflow to infinity in arithmetic. Inputs that produce intermediate Inf or NaN typically cause the line search to fail, returning INFO_ABNORMAL_LNSRLB. The user should ensure f_eval and g_eval return finite values for feasible inputs.

Step-length floor

The line search has an implicit minimum step set by stpmin = 0. After the bracket has shrunk so that stx ~= sty, dcsrch returns with the best step found so far rather than continuing to shrink. See dcsrch.md.

BLAS / LAPACK policy

The algorithm calls these BLAS-level-1 and LAPACK routines:

Call	Used in	Purpose
dcopy	many	Copy a vector.
daxpy	many	y <- alpha*x + y.
dscal	many	y <- alpha*y.

Call	Used in	Purpose
ddot	many	'x' y. \ilinebr </td> </tr></table> |dnrm2| line search, gradients ||x||_2. | |dgemv|formk, subsm, cauchy|y <- alpha*A*x + beta*y. | |dtrsm|bmv, subsm| Triangular solveA x = b. | |dpotrf|formt, formk` | Cholesky factorization. |

Reference build for bit-for-bit conformance

Bit-for-bit reproducibility across BLAS implementations is **not achievable**: OpenBLAS, MKL, Accelerate, and netlib-BLAS differ in internal reduction order and SIMD behavior, and threaded versions add nondeterminism through dynamic chunking.
For `--strict` conformance, ports and the F77 reference must link against the **same reference BLAS/LAPACK build**:

- **Reference BLAS**: netlib BLAS, single-threaded, source build.
- **Reference LAPACK**: netlib LAPACK, single-threaded, source build.
- **Reference compiler**: `gfortran` (any version >= 9 in CI).
- **Reference flags**: `-O2 -fno-fast-math -fno-associative-math -frounding-math` to disable algebraic reordering and float-contract.

CI pins these versions and rebuilds them in the conformance job. The exact version pin is in `.github/workflows/test.yml` (see Phase E of the plan); the spec runner reads the pin from a metadata file in `docs/spec/` so it stays in sync.

Tolerance mode for non-reference BLAS

Ports that must use a non-reference BLAS (e.g., a performance-optimized vendor implementation in production) can run the conformance suite in `--tolerance` mode, which uses per-subroutine numerical tolerances instead of bit-equality. See `07_↩ conformance.md` for the per-routine tolerances.
The `--tolerance` mode is for **validation**, not for "real" conformance – a port that only passes `--tolerance` may diverge from the F77 reference under sufficiently long iteration sequences. For production use of a non-reference BLAS, the port must accept this risk; it is not an algorithmic bug, but a consequence of BLAS nondeterminism.

Order-of-operations constraints

For bit-for-bit conformance, **the order of additions and subtractions matters** (floating-point addition is not associative). Per-subroutine specs in `subroutines/` document the operation order explicitly: which loops accumulate in which direction, which sums group with which, etc.
In particular:

- **Reduction direction**: F77 BLAS-1 routines (`ddot`, `dnrm2`, `daxpy`) accumulate in increasing index order. Ports using their language's BLAS binding inherit this from the reference build.
- **Manual accumulations**: where the algorithm computes a sum without a BLAS call (e.g., the inner loops of `cauchy` and `subsm`), the spec specifies the loop order. Reversing is **not allowed** in `--strict` mode.
- **Fused multiply-add (FMA)**: not used in the F77 reference (the flags above disable it). Ports compiled with FMA enabled will produce different bit-level outputs even for `--strict` mode and are non-conformant.

Magic-constants table

Populated by the Phase D deviations audit; see `05_deviations.md` for the full audit notes.

Algorithmic constants (require port to match)

Constant	Site	Value	Origin	Tunable?
ftol	lnsrlb.f,	1.0e-3	**Deviation**	No. Validated

Call	Used in	Purpose

7 Deviations from the published algorithm

This document catalogues every place where the F77 implementation in `src/` deviates from the published algorithms in `docs/algorithm.pdf`, `docs/code.pdf`, and `docs/acm-remark.pdf`. Each deviation is documented with the location, the published value, the implemented value, and the rationale (or "unknown" if the source paper for the specific constant was not located).

A port that wants to reproduce the F77 reference's numerical behavior **must use the implemented values**, not the published ones; otherwise the conformance suite (in `--strict` mode against the reference BLAS) will fail.

Confirmed deviations from upstream L-BFGS-B 3.0

This fork (jonathanschilling/L-BFGS-B) drops four `info` output parameters from internal helper routines (`bmv`, `cmprlb`, `cauchy`, `subsm`) that were left dead by the LINPACK->LAPACK migration. Upstream 3.0 still has them. The signatures of these four routines therefore differ from any other L-BFGS-B fork that retains the LINPACK-era parameters:

```
upstream:  call bmv(m, sy, wt, col, v, p, info)
this fork: call bmv(m, sy, wt, col, v, p)
upstream:  call cmprlb(..., cnstnd, info)
this fork: call cmprlb(..., cnstnd)
upstream:  call cauchy(..., sbgnrm, info, epsmch)
this fork: call cauchy(..., sbgnrm, epsmch)
upstream:  call subsm(..., wn, iprint, info)
this fork: call subsm(..., wn, iprint)
```

The `info` outputs were always `0` once `dtrsl` was replaced by `dtrsm` (which cannot fail on a non-singular factor – guaranteed by `formt` / `formk` Cholesky checks). The dead-branch handling in `mainlb` was removed at the same time.

`formt` and `formk` retain their `info` outputs because they perform real Cholesky factorisations (`dpotrf`) that can legitimately fail.

Confirmed deviations from the published algorithm

1. `ftol = 1.0e-3` (More-Thuente sufficient-decrease)

Where	Value	Reference
Implemented	`ftol = 1.0d-3`	`src/lnsrlb.f`:110
Published (algorithm tech report)	`alpha = 1.0e-4`	More & Thuente 1994 sec.6

The algorithm.pdf tech report and the More-Thuente paper specify `ftol = 1e-4` (the sufficient-decrease constant in the Wolfe conditions). The L-BFGS-B implementation uses `1e-3`, a looser value.

This deviation is documented in the F77 source itself in the comment block of `src/lnsrlb.f` (lines 99-105)↩ : ∗"the looser value 1.0d-3 matches the implementation that ships with Algorithm 778; neither the ACM paper (docs/code.pdf) nor the 2011 remark documents the change explicitly"∗.

Impact: the Wolfe sufficient-decrease test `f(stp) <= f(0) + ftol * stp * g(0)` is satisfied at slightly larger function values with `ftol = 1e-3` than with `1e-4`. In practice the line search accepts step lengths a bit

earlier than the strict paper algorithm would. The integration test bounds in `tests/check_output.py` were tuned against the implemented value; changing `ftol` to `1e-4` would require regenerating those bounds.

For a port: use `1e-3` for `--strict` conformance. A port that prefers the published value can do so but cannot pass strict mode.

Constants that match the papers (no deviation)

These were checked during the audit and do match the published algorithm or its referenced sources.

`gtol = 0.9` (curvature condition)

`src/lnsrlb.f:110`. Matches More & Thuente 1994 sec.6 and the algorithm.pdf tech report (eq 2.6).

`xtol = 0.1` (relative bracket-width tolerance)

`src/lnsrlb.f:110`. Matches More & Thuente 1994 sec.6.

`p66 = 0.66` (bracket safeguarding factor)

`src/dcstep.f:81`, `src/dcsrch.f:111`. From More & Thuente 1994 sec.3, the safeguarded-step heuristic. Not in algorithm.pdf (which doesn't describe the line search in detail); present in code.pdf and the MINPACK-2 reference. Treated as algorithmic, not arbitrary.

`xtrapl = 1.1`, `xtrapu = 4.0` (dcsrch extrapolation factors)

`src/dcsrch.f:113`. From the MINPACK-2 line search (More-Thuente). Inherited by L-BFGS-B; not in algorithm.pdf.

`p5 = 0.5`, `two = 2.0`, `three = 3.0` (interpolation coefficients)

`src/dcstep.f`, `src/dcsrch.f`. Standard cubic / quadratic interpolation; not algorithm-specific.

`0.0d0`, `1.0d0`

Universal numeric literals; no deviation possible.

`big = 1.0e10` (lnsrlb's no-bounds sentinel)

`src/lnsrlb.f:98`. Implementation detail, not algorithmic. Used when `cnstnd = false` to set `stpmx` to a large finite value.

2.0 in cauchy.f:420

```
f2 = f2 + 2.0d0*dibp*wmp - dibp2*wmw
```

Coefficient in the Taylor expansion of `f1, f2` updates across a breakpoint; matches the derivation in algorithm.pdf sec.4 and code.pdf sec.3 (the `2` comes from the cross term in the quadratic).

Numerical safeguards (not algorithmic deviations, but worth noting)

These are scaling clamps or division guards that don't appear in the papers but are implementation necessities. None changes the algorithm on well-conditioned inputs.

f2 = max(epsmch * f2_org, f2) in cauchy.f:423

Floor for the segment-curvature variable to prevent division-by-near-zero in `dtm = -f1 / f2`. Triggered only on numerical noise; the floor is `epsmch ~= 2.22e-16` times the original `f2`, so it is many orders of magnitude smaller than typical values of `f2`.

dr <= epsmch * ddum curvature-condition gate in mainlb.f:566

```
if (dr .le. epsmch*ddum) then
   ! skip L-BFGS update; refresh
endif
```

Caller-side check (in `mainlb`) that gates calls to `matupd`. If the new `(s, y)` pair has curvature `s'y` smaller than `eps` times the relevant scale, the pair is rejected to avoid corrupting the L-BFGS cache. Standard refresh logic; the threshold `epsmch * ddum` keeps the test scale-invariant.

Constants from earlier suspicion that turned out NOT to exist

The original spec planning suggested suspect constants `1.0e-8` and `1.0e-10` somewhere in `cauchy.f` / `subsm.f`. **The audit found no such constants in the F77 source.** Cauchy's only safety floor uses `epsmch * f2_org` (machine-eps-scaled, not a fixed value).

Audit methodology

For each `src/*.f`:

1. `grep` for double-precision literals (`[0-9]\.[0-9]+(d|e)[+-]?[0-9]+`).
2. `grep` for `parameter (` blocks.
3. `grep` for `epsmch` usage to identify scaled tolerances.
4. Cross-reference each non-trivial constant against:
   - `algorithm.pdf` (Byrd/Lu/Nocedal/Zhu 1995) – the original derivation.
   - `code.pdf` (Zhu/Byrd/Nocedal 1997) – the algorithm-as-implemented paper.
   - `acm-remark.pdf` (Morales/Nocedal 2011) – the `subsm` correction.
   - Cited references in the F77 source headers (e.g., More-Thuente 1994 cited in `dcstep.f` and `dcsrch.f`).

If you spot a deviation not listed above, please open an issue or PR against the spec pack – the audit was performed once and may have missed something subtle.

Summary

One documented algorithm-relevant deviation (`ftol = 1e-3`).

All other numerical literals are either:

- algorithmic constants that match the cited reference (More-Thuente for the line-search ones; algorithm.pdf for the rest),
- universal numeric literals (`0`, `1`, `2`, `3`, `0.5`),
- implementation-detail sentinels (`big = 1e10`),
- or `epsmch`-scaled safety floors / gates.

The implementation is, by this measure, faithful to the published algorithm modulo the documented `ftol` exception.

8 Porting from Fortran 77: practical gotchas

The L-BFGS-B reference implementation in `src/` is Fortran 77 (with Doxygen comment markup). Port authors reading the F77 source alongside the spec – to verify behavior or to confirm operation order – need to be aware of F77 conventions that differ from C-family languages.

This document is **not** a Fortran tutorial. It catalogues the specific traps that have bitten ports of similar codes. Each section flags whether the convention matters for *spec conformance* (i.e., a port must reproduce it) or only for *reading* the F77 source correctly.

Array memory layout: column-major

Fortran arrays are **column-major** (Fortran-natural / "first index varies fastest"). C, C++, Java, Rust, Go, NumPy default-row-major.

For a 2D array `A(i, j)` declared `double precision A(M, N)`:

- F77 in-memory order: `A(1,1), A(2,1), ..., A(M,1), A(1,2), ...`
- C-equivalent declaration `double A[N][M]`: in-memory order `A[0][0], A[0][1], ..., A[0][M-1], A[1][0], ...` – **different**.

For the C-equivalent layout (so the ports can index naturally), the declaration order is *reversed*: F77 `A(M, N)` <-> C `A[N][M]`.

Affects spec conformance:

- Operation order in inner loops sometimes depends on memory layout (e.g., a column-by-column accumulation reduces to `daxpy` calls in F77). Per-subroutine specs document the order independent of layout choice.
- BLAS/LAPACK calls assume column-major. Ports using a row-major BLAS binding must transpose accordingly. The standard CBLAS bindings expose a layout flag (`CblasColMajor`/`CblasRowMajor`) – ports must pick `CblasColMajor` to match the F77 reference.

For reading F77 source: when you see `A(i, j)`, mentally translate to "row `i`, column `j`, with column-`j` contiguous in memory". Inner loops over `i` are the cache-friendly direction in F77.

1-based indexing

F77 arrays default to 1-based indices: `A(1)` is the first element, `A(N)` is the last. Loops written as `do i = 1, N` are inclusive on both ends.

Spec convention: this pack uses 1-based indexing in pseudocode to match the F77 source for ease of cross-reference. Ports in 0-based languages (C, Python, Rust, ...) translate to 0-based in the obvious way. Where the index value matters numerically (rare – usually only for diagnostic output), the spec calls it out.

Character strings: `character*60` and trailing-space padding

F77 strings are fixed-length character arrays. `character*60 task` declares a 60-character buffer; assignments shorter than 60 characters are **right-padded with spaces**. Comparisons ('task .eq. 'FG') `match on the full padded buffer, so`'FG' || 58 spaces == 'FG' `(with ||`` denoting concatenation).

The L-BFGS-B convention: the first 1-8 non-space characters are the state code; the remainder is human-readable diagnostic text:

```
'CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL          '
 ^^^^^^^^^^^                                                 (first 11 = state code)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^   (rest = description)
```

In F77 callers, the convention is to test only a prefix: 'task(1:2) .eq. 'FG', task(1:5) .eq. 'NEW_X'`, etc.

For ports:

- Use the language's native string type. Right-padding is a F77 artifact, not part of the algorithm.
- The state codes themselves matter only for the legacy reverse-comm interface (see `08_legacy_↩ reverse_comm.md`). Callback-based ports use `info` codes instead.

Logical type: `.true.` / `.false.`

F77 logicals (`logical :: prjctd`) are not integers, but in-memory representations vary by compiler (gfortran uses `1` for true, `0` for false, in a 4-byte slot). Comparisons use `.eqv.` / `.neqv.`, not `==`.

For ports: use the language's native boolean. No conformance implications.

`save` semantics

F77 local variables can be declared `save`, which gives them static storage with a value persisting across calls. L-BFGS-B uses `save` in `dcsrch` and a few utility routines.

For ports: the reverse-comm `setulb` interface relies on `save` implicitly via the `isave`/`dsave` arrays passed in by the caller. Callback-based ports do not need static storage; the algorithm's state lives in normal local variables of the running optimization function.

In the F77 source, `save` variables are sometimes initialized only once (via `data` statements) and then mutated; ports reading the source should track this carefully. Per-subroutine specs flag any relevant `save` use.

`implicit none` *(or absence thereof)*

F77 has implicit typing: a variable starting with `i`, `j`, `k`, `l`, `m`, or `n` is integer by default; otherwise real. The L-BFGS-B sources **do not** declare `implicit none` (though we now compile with `-fimplicit-none`, see `CMakeLists.txt`).

This means a typo in a F77 variable name silently introduces a new variable rather than producing a compiler error. Two such bugs have been caught in this codebase historically (the `bmv one` bug, the `test_cmprlb theta` bug). Reading the F77 source, double-check that every variable you see is declared somewhere.

For ports: declare all variables. This is automatic in any modern language.

Integer divide

F77 integer division truncates toward zero (`5 / 2 = 2`, `-5 / 2 = -2`). Most modern languages match this; Python's `/` on integers used to do true division (Python 2) and now does true division everywhere (Python 3 – use `//` for truncating).

For ports: use the language's truncating-divide operator (or floor divide for non-negative operands; the difference only matters for negative operands, which L-BFGS-B does not produce in indexing contexts).

Logical operators: no short-circuit evaluation

F77's `.and.` and `.or.` are **not short-circuit**. Both operands are evaluated.

```
if (i .ge. 1 .and. a(i) .gt. 0) ...   ! evaluates a(i) even if i < 1!
```

This rarely produces visible differences in L-BFGS-B because the F77 source is structured to avoid the issue (with explicit nested `if`), but porters reading the source should not assume short-circuit behavior even where intermediate inputs would be invalid.

For ports: use the language's short-circuit operators (`&&`, `||`, `and`, `or`); this is safer and produces equivalent behavior in L-BFGS-B's actual code paths (which are structured for non-SC).

`do` loops with start > end

F77 `do i = start, end, step` where `step > 0` and `start > end` **executes zero iterations** (compatible with C-family `for` loops on this point).

```
do i = 5, 4
   ...          ! loop body NOT executed
end do
```

This is the same as `for(i=5; i<=4; i++)` in C – body never runs. No conformance gotcha; mentioned only because some ancient F77 dialects behaved differently.

`goto` and arithmetic-`if`

F77 makes liberal use of `goto`. L-BFGS-B has many: `cauchy.f` has ~10 labels, `mainlb.f` has ~20. Most are forward gotos that simulate `break` / `continue` of inner loops.

The Doxygen-doc'd `c>` comments and structured indentation usually clarify intent. When in doubt, the per-subroutine specs in `subroutines/` describe the control flow in language-neutral terms.

There are no `assigned goto` or `computed goto` constructs in L-BFGS-B, only labeled gotos.

`common` blocks

L-BFGS-B has **no `common` blocks**; all state is passed explicitly through arguments. Ports do not need to worry about this F77 idiom.

(Some older numerical Fortran code used `common` for shared storage; L-BFGS-B avoids this, which made the F77-to-callback translation in `mainlb` straightforward.)

`equivalence` declarations

L-BFGS-B has **no `equivalence`** statements (which let two F77 variables share the same memory location). Ports do not need to model aliasing.

`data` statement initialization

F77 `data` initializes variables once before execution. Used in L-BFGS-B for constants like `parameter (zero=0.0d0)`. Any modern language's `const` / `static` initialization is equivalent.

Reverse communication (covered separately)

The most language-specific F77 idiom in L-BFGS-B is the reverse-communication protocol of `setulb`. This is captured in `08_legacy_reverse_comm.md` as an optional appendix. It is **not part of the canonical algorithm spec**; ports targeting modern languages should use callbacks instead (see `02_api.md`).

Source-file format: fixed-form columns

The F77 sources in `src/*.f` use **fixed-form Fortran**: columns 1-5 are reserved for labels, column 6 for continuation marker, columns 7-72 for code, columns beyond 72 ignored. The newer drivers in `drivers/*.f90` use free-form.

Port authors do not need to worry about column conventions, but should be aware that:

- A line continuation in F77 fixed form is a non-blank, non-zero character in column 6 (the `+` characters in `mainlb.f`).
- Long F77 expressions span multiple lines via this mechanism; reading them requires recognizing that each `+` line continues the prior.

Doxygen `c>` markup

The L-BFGS-B sources use Doxygen-aware F77 comments: lines starting with `c>` (or `!>` in F90 sources) are treated as Doxygen documentation. Most subroutines have an `@param` block at the top documenting each argument, and a brief `@brief` description. These are the canonical per-parameter docs; the spec's per-subroutine files in `subroutines/` are the canonical *behavioral* docs.

Compiler flags used by this repo

The project's `CMakeLists.txt` sets, for `gfortran`:
`-fimplicit-none -Wall -Wextra -Wno-compare-reals`

Plus, when `LBFGSB_COVERAGE=ON`:
`--coverage -O0 -g`

The `-Wno-compare-reals` is intentional: L-BFGS-B legitimately uses exact floating-point comparison (== / /=) on real values to detect algorithmic states (e.g., variable exactly at a bound, exact unit step). This is checked-and-correct usage; the compiler warning is a stylistic check that does not apply here. Per-subroutine specs flag each such comparison and explain why it's correct.

For bit-for-bit conformance, additionally:
`-O2 -fno-fast-math -fno-associative-math -frounding-math`

(see `04_numerics.md`).

9 Conformance criteria

A port is **conformant** if it passes `docs/spec/runner/conformance.py` in **strict** mode against `docs/spec/data/`. Strict mode requires exact IEEE-754 byte equality on every output, given the same input.

For ports that cannot use the reference BLAS / LAPACK (and therefore cannot achieve bit-for-bit numerical agreement with the F77 reference; see `04_numerics.md`), the runner provides a **tolerance** mode using per-subroutine numerical tolerances.

Engines

The conformance runner supports two implementations to validate against the JSON test vectors:

- `**--engine python**` (default). Validates the Python+NumPy reference implementation in `docs/spec/reference_impl/`. Uses `scipy.linalg` for triangular solves and Cholesky.
- `**--engine fortran**`. Subprocess-calls `tests/conformance/conformance_driver` (built by CMake) which links `liblbfgsb.so` and runs the actual F77 routines. This is the strongest validation: it proves the JSON test vectors match the canonical F77 implementation directly, not just transitively.

Both engines accept both modes (`--strict` and `--tolerance`).

Modes

`--strict` (default)

```
python3 docs/spec/runner/conformance.py --strict --engine fortran
```

Each output value must match `np.float64(expected).tobytes() == np.float64(actual).tobytes()`. Integer and string values must match exactly; boolean values must match exactly; arrays must have matching length and pointwise bit-equal entries.

The repo's expected end state:

Engine	–strict	–tolerance
`fortran`	101/101	101/101
`python`	100/101 (`bmv_case_3` BLAS variance)	101/101

Strict mode is the gold-standard conformance test. It requires:

- The reference BLAS / LAPACK pin (see `04_numerics.md`).
- IEEE-754 round-to-nearest with no extended precision.
- Operation order matching the F77 source (documented per subroutine).

`--tolerance`

```
python3 docs/spec/runner/conformance.py --tolerance
```

Output values are compared with absolute + relative tolerance per subroutine (table below). Integer / string / boolean comparisons stay exact.

This mode is for ports that:

- Use a non-reference BLAS for performance (MKL, OpenBLAS-threaded, ...).
- Use language-native linear-algebra primitives (Eigen, NumPy with default BLAS, LinearAlgebra.jl, ...).

A port that only passes `--tolerance` may diverge from the F77 reference under sufficiently long iteration sequences. This is not a bug – it's the natural consequence of BLAS non-determinism. For most applications this is acceptable; for bit-reproducible scientific computing it is not.

Per-subroutine tolerances

Used in `--tolerance` mode. Match constants in `runner/conformance.py:TOLERANCES`.

Subroutine	abs	rel	Justification
`projgr`	0	0	Pure min/max/abs; integer arithmetic effectively.
`errclb`	0	0	Integer / string only.
`active`	0	0	Bound projection is exact.
`freev`	0	0	Integer / boolean.
`hpsolb`	0	0	Comparisons + assignments.
`bmv`	1e-14	1e-14	Triangular solves (small ULP from BLAS).
`cmprlb`	1e-14	1e-14	Calls `bmv`; inherits its tolerance.
`formt`	1e-14	1e-14	Cholesky factor (LAPACK).
`matupd`	0	0	Pure dot products at the BLAS level – F77 inner-product order is used.
`dcstep`	1e-14	1e-14	Cubic / secant interpolation.
`formk`	1e-14	1e-14	Two Choleskys + triangular-solve block.
`dcsrch`	1e-14	1e-14	Full line-search arithmetic.
`lnsrlb`	1e-14	1e-14	min/max + dot + projection.
`cauchy`	1e-13	1e-13	Accumulated `f1, f2` updates over breakpoints.
`subsm`	1e-13	1e-13	Subspace solve via `wn` factor.
`mainlb`	1e-12	1e-12	Full algorithm loop; tolerances accumulate.

How a port reports conformance

The port runs the conformance runner against its own implementation (see `runner/README.md` for the I/O protocol). Output:

```
=== Conformance (strict) ===
  Pass: 101
  Fail: 0
  Skip: 0
```

For CI integration, the runner exits with status `0` on full pass, `1` on any failure. A port that passes strict mode against the reference BLAS pin earns the conformance badge; ports that pass only tolerance mode are noted as such.

Known caveats

- `**bmv_case_3` is BLAS-sensitive`**`. The JSON expected values are the actual output of the F77 reference linked against OpenBLAS (the reference build for this repo). The Python reference impl (linked through `scipy.linalg.solve_triangular`) produces a 1-ULP- different value for `p[2]`. Strict mode therefore passes against the F77 build (`--engine fortran`) but fails for one case against the Python ref (`--engine python`). Tolerance mode passes for both engines. This is a real BLAS-reproducibility effect, not a defect: scipy's triangular-solve has a slightly different reduction order than netlib/OpenBLAS `dtrsm`. Ports using the reference BLAS pin will pass strict mode; ports using a different BLAS may need to use tolerance mode.
- **Trajectory cases for `dcsrch` and `lnsrlb`** (full line searches) are not yet encoded as JSON. They require a multi-call replay protocol and are deferred to a future Phase C extension. Until then, line-search behavior is validated only at the single-call granularity (the 8 + 7 cases that exercise input validation and per-call branches).
- `**mainlb` integration cases`**` (full optimization runs) are similarly deferred. The Python reference impl is verified to converge on a quadratic and on bound-constrained Rosenbrock; full end-to-end JSON test vectors covering convergence trajectories are a future extension.

10 Appendix: F77 reverse-communication interface

> **This appendix is OPTIONAL.** It is needed only by ports that want to expose the F77 `setulb` ABI for compatibility with existing callers. Ports targeting modern languages should use the callback-based interface in `02_api.md` instead.

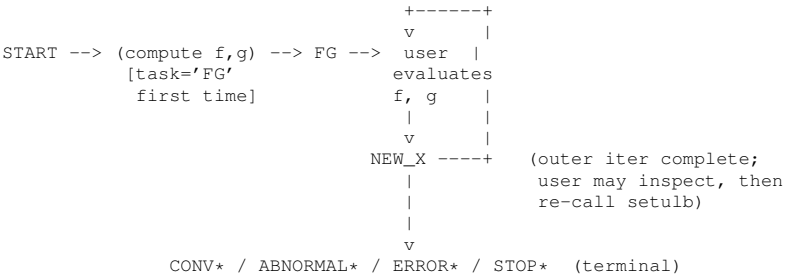
The F77 reference implementation exposes the optimizer through a *reverse-communication* protocol: instead of the user passing function and gradient callbacks, the user calls `setulb` repeatedly; each call returns control with a `task` string indicating what the user should do next (typically: "evaluate `f` and `g` at the current `x`, then call back").

Reverse communication exists because Fortran 77 lacks first-class function values. Modern languages (including Fortran 2003 and later) have closures, so most ports do not need this protocol. Use this appendix only if:

- Existing F77 / scientific code calls `setulb` and you want a drop-in replacement, or
- Your language has restricted callback support (some embedded / RT-Java environments, certain FPGA toolchains).

Protocol overview

State diagram



Task strings

`task` is a `character*60` buffer. The first 1-8 non-space characters are the state code; the rest is human-readable description. Test state with prefix matching: 'task(1:2) == 'FG', task(1:5) == 'NEW_X'`, etc.

task prefix	**When set**	**Meaning**
'START'\ilinebr </td> <td class="markdownTableBody↩ None"> By user before first call \ilinebr </td> <td class="markdownTableBodyNone"> Initialize a new optimization. \ilinebr </td> </tr> <tr class="markdownTableRowEven"> <td class="markdown↩ TableBodyNone">'FG'\ilinebr </td> <td class="markdown↩ TableBodyNone"> Bysetulbon return \ilinebr </td> <td class="markdownTableBodyNone"> User must computefandgat the currentxand callsetulbagain. \ilinebr </td> </tr> <tr class="markdownTableRowOdd"> <td class="markdownTableBodyNone">'NEW_X'\ilinebr </td> <td class="markdownTableBodyNone"> Bysetulbon return \ilinebr </td> <td class="markdownTableBodyNone"> A new iterate is available inx; the user may inspect, log, etc., then callsetulbagain to continue iterating. \ilinebr </td> </tr> <tr class="markdownTableRowEven"> <td class="markdownTableBodyNone">'CONV'...\ilinebr </td> <td class="markdownTableBodyNone"> Bysetulbon return \ilinebr </td> <td class="markdownTable↩ BodyNone"> Convergence reached. Specific suffix indicates which test fired. Terminal. \ilinebr </td> </tr> <tr class="markdownTableRowOdd"> <td class="markdownTableBodyNone">'ABNORMAL'...\ilinebr </td> <td class="markdownTableBodyNone"> Bysetulbon return \ilinebr </td> <td class="markdownTableBodyNone"> Line search failed or other abnormal termination. Terminal. \ilinebr </td> </tr> <tr class="markdownTableRowEven"> <td class="markdownTableBodyNone">'ERROR'...\ilinebr </td> <td class="markdownTableBodyNone"> Bysetulbon return \ilinebr </td> <td class="markdownTableBody↩ None"> Input validation failed. Terminal. \ilinebr </td> </tr> <tr class="markdownTableRowOdd"> <td class="markdownTableBodyNone">'STOP'\ilinebr </td> <td class="markdownTableBodyNone"> By user any time \ilinebr </td> <td class="markdownTableBodyNone"> Request immediate termination.setulbwill tidy up and return with a terminaltask`.		

Specific terminal task strings:

```
'CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL'
'CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH'
'ABNORMAL TERMINATION IN LNSRLB'
'ERROR: N <= 0'
'ERROR: M <= 0'
'ERROR: FACTR < 0'
'ERROR: INVALID NBD'
'ERROR: NO FEASIBLE SOLUTION'
'STOP: CPU EXCEEDING THE TIME LIMIT.'
```

(See `subroutines/errclb.md` for the full input-validation case list.)

Calling sequence (worked example)

```
character*60 :: task, csave
logical      :: lsave(4)
integer      :: isave(44), iwa(3*n)
real(8)      :: dsave(29), wa(2*m*n + 5*n + 11*m*m + 8*m)
real(8)      :: x(n), l(n), u(n), g(n), f, factr, pgtol
integer      :: nbd(n), iprint
! Initial setup
x = ...         ! initial point
l = ...; u = ...; nbd = ...
factr = 1.0d7
pgtol = 1.0d-5
iprint = -1
task = 'START'
do
  call setulb(n, m, x, l, u, nbd, f, g, factr, pgtol, &
              wa, iwa, task, iprint, csave, lsave, isave, dsave)
  if (task(1:2) == 'FG') then
    f = ...      ! evaluate f(x)
    g = ...      ! evaluate gradient at x
    cycle        ! call setulb again with the new f, g
  else if (task(1:5) == 'NEW_X') then
    ! a new iterate is in x; optionally inspect, then continue
    cycle
  else
    exit         ! terminal task: CONV / ABNORMAL / ERROR / STOP
  end if
end do
! task contains the termination message; isave/dsave have stats
```

Workspace arrays

`setulb` requires the user to provide all storage; nothing is allocated internally. Sizes:

Array	Type	Size	Purpose
`wa`	`real(8)`	`2*m*n + 5*n + 11*m^2 + 8*m`	All double-precision working storage.
`iwa`	`integer`	`3*n`	All integer working storage (`index`, `iwhere`, `indx2`).
`task`	`character*60`	–	Reverse-comm state code.
`csave`	`character*60`	–	Inner reverse-comm state for `dcsrch`.
`lsave`	`logical`	`4`	`prjctd`, `cnstnd`, `boxed`, `updatd`.
`isave`	`integer`	`44`	Saved integer state across calls.
`dsave`	`real(8)`	`29`	Saved real state across calls.

The user must not modify any of these between calls except `f` and `g` when ‘task = 'FG', `and`task`itself `for`'START'`or`'STOP'`.

wa partitioning

Set during the "START"call bysetulb; remains stable across all subsequent calls. The first 16 slots ofisave` hold the offsets:

isave[k]	Stores	Logical object	F77 name
1	m*n	(size constant)	–
2	m^2	(size constant)	–
3	4*m^2	(size constant)	–
4	offset	S matrix (history of s-vectors)	ws
5	offset	Y matrix (history of y-vectors)	wy
6	offset	sy (S'Y packed: D + L)	sy
7	offset	ss (S'S upper triangle)	ss
8	offset	T (Cholesky factor)	wt
9	offset	K (2m * 2m reduced Hessian)	wn
10	offset	K_chol (Cholesky of K)	snd
11	offset	xc (Cauchy point)	z
12	offset	r (reduced gradient)	r
13	offset	d (search direction)	d
14	offset	t (scratch)	t
15	offset	xp (previous iterate)	xp
16	offset	scratch (8*m)	wa (inner)

All offsets are 1-based positions within the user's wa array.

iwa partitioning

Set deterministically by setulb's call to mainlb:

iwa[k]	Logical object
iwa[1..n]	index (free-variable permutation)
iwa[n+1..2n]	iwhere (per-variable bound status)
iwa[2n+1..3n]	indx2 (entering/leaving lists)

lsave semantics

Available on exit with 'task = 'NEW_X'`:

lsave[k]	Meaning
1	prjctd: initial x0 was infeasible and projected.
2	cnstnd: at least one variable has a bound.
3	boxed: every variable has both lower and upper bounds.
4	updatd: L-BFGS history was updated in this iteration.

`isave` saved state

Beyond the workspace offsets in `1..16`, `isave` carries diagnostic counters and saved internal state. The slots documented for users (available on exit with ‘task = 'NEW_X'`):

`isave[k]`	Meaning
`22`	Total intervals explored across all Cauchy point searches.
`26`	Total skipped BFGS updates before the current iteration.
`30`	Iteration counter (`iter`).
`31`	Total BFGS updates *prior to* the current iteration.
`33`	Cauchy intervals explored in the current iteration.
`34`	Total `f` and `g` evaluations (cumulative).
`36`	`f`/`g` evaluations in the current iteration's line search.
`37`	Subspace-argmin location flag: `0` if within box, `1` if beyond box (clipped).
`38`	Number of free variables in the current iteration.
`39`	Number of active constraints in the current iteration.
`40`	Sentinel `n + 1 - isave[40]` = number of variables that *left* the active set this iteration.
`41`	Number of variables that *entered* the active set this iteration.

Slots 17-21, 23-25, 27-29, 32, 35, 42-44 are private to `mainlb` and its callees (saving control-flow state for resume-after-“FG`); the user must not read or modify them.

`dsave` saved state

Available on exit with ‘task = 'NEW_X'`:

`dsave[k]`	Meaning
`1`	Current `theta`.
`2`	`f` at the previous iterate.
`3`	`factr * epsmch`.
`4`	2-norm of the search direction `d`.
`5`	`epsmch` as computed by `mainlb`.
`7`	Accumulated wallclock spent on Cauchy point.
`8`	Accumulated wallclock on subspace minimization.
`9`	Accumulated wallclock on line search.
`11`	Slope of `f` along `d` at the current line-search point.
`12`	Maximum relative step length imposed by the box geometry.

| `13` | `\|\|g_proj\|\|_Inf`. | | `14` | Relative step length accepted in the line search. | | `15` | Slope of `f` along `d` at the start of the line search. | | `16` | Square of the 2-norm of `d`. |

Slots 6, 10, 17-29 are private to internal routines.

`csave` saved state

Holds the inner reverse-comm state for the More-Thuente line search (`dcsrch`). The user must not modify it. When the line search needs another `f`/`g` evaluation, the outer state machine in `mainlb` sets ‘task = 'FG'`and the user provides values;` dcsrch `resumes from the state captured in` csave`.

Implementation notes for ports

A port that exposes this protocol must:

1. **On 'task = 'START'**: validate inputs (pererrclb.md<tt>), partition wa/iwa<tt>(or whatever storage equivalent the port uses), do the initial projection, and yield withtask = 'FG'requesting the firstf/g.
2. **On subsequent calls**: dispatch on the previoustaskvalue (or on a hidden program-counter saved inisave) to resume the correct internal location. The F77 source uses agoto-based state machine; ports withoutgoto` typically use a switch on a saved enum.
3. **Preserve all working state** in isave/dsave/lsave/csave between calls. The user is allowed to inspect the documented slots; the port must not move them.
4. **On 'task = 'STOP'`**: the user can set this any time. The port tidies up (no work in progress to abort cleanly) and returns with 'task = 'STOP: <reason>`' as a terminal state.

Mapping back to the callback interface

A port that already implements the canonical callback API in 02_api.md can wrap it in a reverse-comm shim by:

1. Running the callback-based optimizer **on a separate stack** / **coroutine** / **thread** that yields when the callback is invoked.
2. The shim's setulb-equivalent function calls resume() on the coroutine; when the coroutine yields with the trial x, the shim sets 'task = 'FG'and returns to the user.
3. On the next call, the shim writes the user-suppliedfandginto the coroutine's communication slot andresume()`s again.

Most languages support this shape via:

- Python: generators (yield) or asyncio coroutines.
- C/C++: setjmp/longjmp, fibers, or std::generator (C++23).
- Rust: Generator trait or hand-rolled state machine.
- Go: goroutines + channels.
- Java: Thread.suspend/resume (deprecated; use a worker thread with synchronization), or virtual threads (Loom).

Ports that need the legacy ABI but can't use coroutines must implement the protocol natively as the F77 source does – saving program-counter and local state in isave/dsave. This is more work than the coroutine wrapper but has no extra runtime cost.

Testing

The conformance suite includes JSON test vectors that exercise the reverse-comm protocol end-to-end (driving the optimizer through full runs of the integration drivers). These vectors test the legacy interface only; ports that don't implement this appendix can skip them.

11 File Index

11.1 File List

Here is a list of all documented files with brief descriptions:

12 File Documentation

12.1 src/active.f File Reference

Functions/Subroutines

- subroutine active (n, l, u, nbd, x, iwhere, iprint, prjctd, cnstnd, boxed)

  This subroutine initializes iwhere and projects the initial x to the feasible set if necessary.

12.1.1 Function/Subroutine Documentation

12.1.1.1 active()

```
subroutine active (
        integer n,
        double precision, dimension(n) l,
        double precision, dimension(n) u,
        integer, dimension(n) nbd,
        double precision, dimension(n) x,
        integer, dimension(n) iwhere,
        integer iprint,
        logical prjctd,
        logical cnstnd,
        logical boxed )
```

This subroutine initializes iwhere and projects the initial x to the feasible set if necessary.

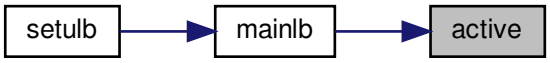
Parameters

n	number of parameters
l	lower bounds on parameters
u	upper bounds on parameters
nbd	indicates which bounds are present
x	position
iwhere	On entry iwhere is unspecified. On exit: iwhere(i)= • -1 if x(i) has no bounds • 3 if l(i)=u(i), • 0 otherwise. In cauchy, iwhere is given finer gradations.
iprint	console output flag
prjctd	On exit .true. if any input x(i) had to be clipped onto its bound (the user supplied an infeasible starting point).
cnstnd	On exit .true. if any variable has at least one bound (any nbd(i) /= 0); .false. for purely unconstrained problems.
boxed	On exit .true. iff every variable has both lower and upper bounds (every nbd(i) == 2); .false. otherwise.

Definition at line 35 of file active.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.2 src/bmv.f File Reference

Functions/Subroutines

- subroutine bmv (m, sy, wt, col, v, p)

  This subroutine computes the product of the 2m x 2m middle matrix in the compact L-BFGS formula of B and a 2m vector v.

12.2.1 Function/Subroutine Documentation

12.2.1.1 bmv()

```
subroutine bmv (
    integer m,
    double precision, dimension(m, m) sy,
    double precision, dimension(m, m) wt,
    integer col,
    double precision, dimension(2*col) v,
    double precision, dimension(2*col) p )
```

This subroutine computes the product of the 2m x 2m middle matrix in the compact L-BFGS formula of B and a 2m vector v; it returns the product in p.

Parameters

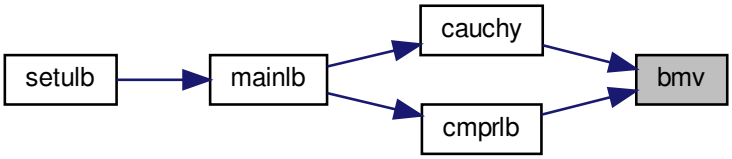
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
sy	On entry sy specifies the matrix S'Y. On exit sy is unchanged.
wt	On entry wt specifies the upper triangular matrix J' which is the Cholesky factor of (thetaS'S+LD^(-1)L'). On exit wt is unchanged.
col	On entry col specifies the number of s-vectors (or y-vectors) stored in the compact L-BFGS formula. On exit col is unchanged.
v	On entry v specifies vector v. On exit v is unchanged.
p	On entry p is unspecified. On exit p is the product Mv.

Historical note: this routine used to take an `info` output parameter for the LINPACK `dtrsl` triangular-solve return status. Since LAPACK's `dtrsm` replacement cannot fail on a non-singular triangular factor (and `formt` ensures `wt` is non-singular), the parameter was always 0 on exit and has been removed.

Definition at line 36 of file bmv.f.

Referenced by cauchy(), and cmprlb().

Here is the caller graph for this function:



12.3 src/cauchy.f File Reference

Functions/Subroutines

- subroutine cauchy (n, x, l, u, nbd, g, iorder, iwhere, t, d, xcp, m, wy, ws, sy, wt, theta, col, head, p, c, wbp, v, nseg, iprint, sbgnrm, epsmch)

  Compute the Generalized Cauchy Point along the projected gradient direction.

12.3.1 Function/Subroutine Documentation

12.3.1.1 cauchy()

```
subroutine cauchy (
        integer n,
        double precision, dimension(n) x,
        double precision, dimension(n) l,
        double precision, dimension(n) u,
        integer, dimension(n) nbd,
        double precision, dimension(n) g,
        integer, dimension(n) iorder,
        integer, dimension(n) iwhere,
        double precision, dimension(n) t,
        double precision, dimension(n) d,
        double precision, dimension(n) xcp,
        integer m,
        double precision, dimension(n, col) wy,
        double precision, dimension(n, col) ws,
        double precision, dimension(m, m) sy,
        double precision, dimension(m, m) wt,
        double precision theta,
```

```
integer col,
integer head,
double precision, dimension(2*m) p,
double precision, dimension(2*m) c,
double precision, dimension(2*m) wbp,
double precision, dimension(2*m) v,
integer nseg,
integer iprint,
double precision sbgnrm,
double precision epsmch )
```

For given x, l, u, g (with sbgnrm $>$ 0), and a limited memory BFGS matrix B defined in terms of matrices WY, WS, WT, and scalars head, col, and theta, this subroutine computes the generalized Cauchy point (GCP), defined as the first local minimizer of the quadratic

```
Q(x + s) = g's + 1/2 s'Bs
```

along the projected gradient direction P(x-tg,l,u). The routine returns the GCP in xcp.

Parameters

n	On entry n is the dimension of the problem. On exit n is unchanged.
x	On entry x is the starting point for the GCP computation. On exit x is unchanged.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, and • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
g	On entry g is the gradient of f(x). g must be a nonzero vector. On exit g is unchanged.
iorder	iorder will be used to store the breakpoints in the piecewise linear path and free variables encountered. On exit, • iorder(1),...,iorder(nleft) are indices of breakpoints which have not been encountered; • iorder(nleft+1),...,iorder(nbreak) are indices of encountered breakpoints; and • iorder(nfree),...,iorder(n) are indices of variables which have no bound constraits along the search direction.

Parameters

iwhere	On entry iwhere indicates only the permanently fixed (iwhere=3) or free (iwhere= -1) components of x. On exit iwhere records the status of the current x variables. iwhere(i)= • -3 if x(i) is free and has bounds, but is not moved • 0 if x(i) is free and has bounds, and is moved • 1 if x(i) is fixed at l(i), and l(i) .ne. u(i) • 2 if x(i) is fixed at u(i), and u(i) .ne. l(i) • 3 if x(i) is always fixed, i.e., u(i)=x(i)=l(i) • -1 if x(i) is always free, i.e., it has no bounds.
t	working array; will be used to store the break points.
d	the Cauchy direction P(x-tg)-x
xcp	returns the GCP on exit
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
sy	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wt	On entry this stores the Cholesky factorization of (theta∗S'S+LD^(-1)L'), that defines the limited memory BFGS matrix. On exit this array is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
head	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
p	will be used to store the vector p = W^(T)d.
c	will be used to store the vector c = W^(T)(xcp-x).
wbp	will be used to store the row of W corresponding to a breakpoint.
v	working array
nseg	On exit nseg records the number of quadratic segments explored in searching for the GCP.

Parameters

iprint	variable that must be set by the user. It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
sbgnrm	On entry sbgnrm is the norm of the projected gradient at x. On exit sbgnrm is unchanged.
epsmch	machine precision epsilon

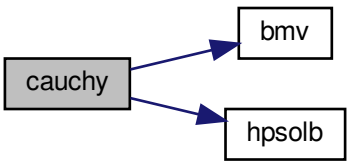
Historical note: this routine used to take an `info` output parameter to forward errors from the embedded `bmv` calls. Since `bmv` cannot fail under LAPACK `dtrsm`, the parameter was always 0 on exit and has been removed.

Definition at line 131 of file cauchy.f.

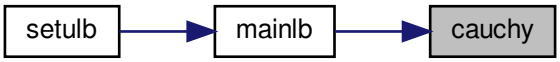
References bmv(), and hpsolb().

Referenced by mainlb().

Here is the call graph for this function:



Here is the caller graph for this function:



12.4 src/cmprlb.f File Reference

Functions/Subroutines

- subroutine cmprlb (n, m, x, g, ws, wy, sy, wt, z, r, wa, index, theta, col, head, nfree, cnstnd)

  This subroutine computes r=-Z'B(xcp-xk)-Z'g by using wa(2m+1)=W'(xcp-x) from subroutine cauchy.

12.4.1 Function/Subroutine Documentation

12.4.1.1 cmprlb()

```
subroutine cmprlb (
    integer n,
    integer m,
    double precision, dimension(n) x,
    double precision, dimension(n) g,
    double precision, dimension(n, m) ws,
    double precision, dimension(n, m) wy,
    double precision, dimension(m, m) sy,
    double precision, dimension(m, m) wt,
    double precision, dimension(n) z,
    double precision, dimension(n) r,
    double precision, dimension(4*m) wa,
    integer, dimension(n) index,
    double precision theta,
    integer col,
    integer head,
    integer nfree,
    logical cnstnd )
```

This subroutine computes r=-Z'B(xcp-xk)-Z'g by using wa(2m+1)=W'(xcp-x) from subroutine cauchy.

Parameters

n	number of parameters
m	history size of Hessian approximation
x	position
g	gradient
ws	part of L-BFGS matrix
wy	part of L-BFGS matrix
sy	part of L-BFGS matrix
wt	part of L-BFGS matrix
z	The generalized Cauchy point xcp computed by cauchy. Used here as the linearisation point: r encodes -B(z-x) - g restricted to the free variables.
r	On exit r(1:nfree) contains -theta∗(z(k)-x(k)) - g(k) plus the W∗M^{-1}∗W' correction, where k = index(i). Caller uses this as the residual for the subspace minimisation problem. When cnstnd=.false. and col>0, the shortcut path sets r(1:n) = -g(:) without using the index array.
wa	Length-4m workspace shared with cauchy. On entry, the segment wa(2m+1 : 2m+2col) holds W'(z-x) (filled by cauchy). On exit wa(1 : 2col) holds M^{-1}∗W'(z-x) from the bmv call.
index	Permutation of (1..n): index(1..nfree) lists the indices of variables that are free at the GCP and are the active optimisation variables here.
theta	Scaling factor specifying the initial Hessian B_0 = theta∗I.

Parameters

col	Number of stored (s,y) correction pairs (0 on the first iteration; up to m thereafter).
head	Index in the cyclic WS/WY buffer of the oldest stored correction. Used to walk the columns in chronological order.
nfree	Number of free variables; size of the subspace problem.
cnstnd	.true. if the problem has bounds; controls the shortcut path described under
r.	Historical note: this routine used to take an `info` output parameter to forward errors from the embedded `bmv` call. Since `bmv` cannot fail under LAPACK `dtrsm`, the parameter was always 0 on exit and has been removed.

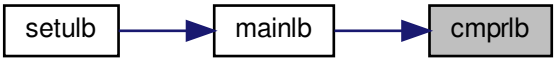
Definition at line 44 of file cmprlb.f.

References bmv().

Referenced by mainlb().

Here is the call graph for this function:

Here is the caller graph for this function:



12.5 src/dcsrch.f File Reference

Functions/Subroutines

- subroutine dcsrch (f, g, stp, ftol, gtol, xtol, stpmin, stpmax, task, isave, dsave)

  This subroutine finds a step that satisfies a sufficient decrease condition and a curvature condition.

12.5.1 Function/Subroutine Documentation

12.5.1.1 dcsrch()

```
subroutine dcsrch (
        double precision f,
        double precision g,
        double precision stp,
        double precision ftol,
        double precision gtol,
        double precision xtol,
        double precision stpmin,
        double precision stpmax,
        character*(*) task,
        integer, dimension(2) isave,
        double precision, dimension(13) dsave )
```

This subroutine finds a step that satisfies a sufficient decrease condition and a curvature condition.

Each call of the subroutine updates an interval with endpoints stx and sty. The interval is initially chosen so that it contains a minimizer of the modified function

```
psi(stp) = f(stp) - f(0) - ftol*stp*f'(0).
```

If psi(stp) <= 0 and f'(stp) >= 0 for some step, then the interval is chosen so that it contains a minimizer of f.

The algorithm is designed to find a step that satisfies the sufficient decrease condition

```
f(stp) <= f(0) + ftol*stp*f'(0),
```

and the curvature condition

```
abs(f'(stp)) <= gtol*abs(f'(0)).
```

If ftol is less than gtol and if, for example, the function is bounded below, then there is always a step which satisfies both conditions.

If no step can be found that satisfies both conditions, then the algorithm stops with a warning. In this case stp only satisfies the sufficient decrease condition.

A typical invocation of dcsrch has the following outline:

```
   task = 'START'
10 continue
   call dcsrch( ... )
   if (task .eq. 'FG') then
     evaluate the function and the gradient at stp
   goto 10
   end if
```

NOTE: The user must no alter work arrays between calls.

Parameters

f	On initial entry f is the value of the function at 0. On subsequent entries f is the value of the function at stp. On exit f is the value of the function at stp.
g	On initial entry g is the derivative of the function at 0. On subsequent entries g is the derivative of the function at stp. On exit g is the derivative of the function at stp.
stp	On entry stp is the current estimate of a satisfactory step. On initial entry, a positive initial estimate must be provided. On exit stp is the current estimate of a satisfactory step if task = 'FG'. If task = 'CONV' then stp satisfies the sufficient decrease and curvature condition.

Parameters

ftol	On entry ftol specifies a nonnegative tolerance for the sufficient decrease condition. On exit ftol is unchanged.
gtol	On entry gtol specifies a nonnegative tolerance for the curvature condition. On exit gtol is unchanged.
xtol	On entry xtol specifies a nonnegative relative tolerance for an acceptable step. The subroutine exits with a warning if the relative difference between sty and stx is less than xtol. On exit xtol is unchanged.
stpmin	On entry stpmin is a nonnegative lower bound for the step. On exit stpmin is unchanged.
stpmax	On entry stpmax is a nonnegative upper bound for the step. On exit stpmax is unchanged.
task	On initial entry task must be set to 'START'. On exit task indicates the required action: • If task(1:2) = 'FG' then evaluate the function and derivative at stp and call dcsrch again. • If task(1:4) = 'CONV' then the search is successful. • If task(1:4) = 'WARN' then the subroutine is not able to satisfy the convergence conditions. The exit value of stp contains the best point found during the search. • If task(1:5) = 'ERROR' then there is an error in the input arguments. On exit with convergence, a warning or an error, the variable task contains additional information.
isave	work array
dsave	work array

Definition at line 94 of file dcsrch.f.

References dcstep().

Referenced by lnsrlb().

Here is the call graph for this function:

Here is the caller graph for this function:

12.6 src/dcstep.f File Reference

Functions/Subroutines

- subroutine dcstep (stx, fx, dx, sty, fy, dy, stp, fp, dp, brackt, stpmin, stpmax)

  This subroutine computes a safeguarded step for a search procedure and updates an interval that contains a step that satisfies a sufficient decrease and a curvature condition.

12.6.1 Function/Subroutine Documentation

12.6.1.1 dcstep()

```
subroutine dcstep (
        double precision stx,
        double precision fx,
        double precision dx,
        double precision sty,
        double precision fy,
        double precision dy,
        double precision stp,
        double precision fp,
        double precision dp,
        logical brackt,
        double precision stpmin,
        double precision stpmax )
```

This subroutine computes a safeguarded step for a search procedure and updates an interval that contains a step that satisfies a sufficient decrease and a curvature condition.

The parameter stx contains the step with the least function value. If brackt is set to .true. then a minimizer has been bracketed in an interval with endpoints stx and sty. The parameter stp contains the current step. The subroutine assumes that if brackt is set to .true. then

```
min(stx,sty) < stp < max(stx,sty),
```

and that the derivative at stx is negative in the direction of the step.

Parameters

stx	On entry stx is the best step obtained so far and is an endpoint of the interval that contains the minimizer. On exit stx is the updated best step.
fx	On entry fx is the function at stx. On exit fx is the function at stx.
dx	On entry dx is the derivative of the function at stx. The derivative must be negative in the direction of the step, that is, dx and stp - stx must have opposite signs. On exit dx is the derivative of the function at stx.
sty	On entry sty is the second endpoint of the interval that contains the minimizer. On exit sty is the updated endpoint of the interval that contains the minimizer.
fy	On entry fy is the function at sty. On exit fy is the function at sty.
dy	On entry dy is the derivative of the function at sty. On exit dy is the derivative of the function at the exit sty.

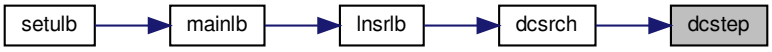
Parameters

stp	On entry stp is the current step. If brackt is set to .true. then on input stp must be between stx and sty. On exit stp is a new trial step.
fp	On entry fp is the function at stp. On exit fp is unchanged.
dp	On entry dp is the the derivative of the function at stp. On exit dp is unchanged.
brackt	On entry brackt specifies if a minimizer has been bracketed. Initially brackt must be set to .false. On exit brackt specifies if a minimizer has been bracketed. When a minimizer is bracketed brackt is set to .true.
stpmin	On entry stpmin is a lower bound for the step. On exit stpmin is unchanged.
stpmax	On entry stpmax is an upper bound for the step. On exit stpmax is unchanged.

Definition at line 66 of file dcstep.f.

Referenced by dcsrch().

Here is the caller graph for this function:



12.7 src/errclb.f File Reference

Functions/Subroutines

- subroutine errclb (n, m, factr, l, u, nbd, task, info, k)

  This subroutine checks the validity of the input data.

12.7.1 Function/Subroutine Documentation

12.7.1.1 errclb()

```
subroutine errclb (
    integer n,
    integer m,
    double precision factr,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    integer, dimension(n) nbd,
    character*60 task,
    integer info,
    integer k )
```

This subroutine checks the validity of the input data.

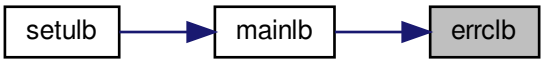
Parameters

n	number of parameters
m	history size of approximated Hessian
factr	convergence criterion on function value
l	lower bounds for parameters
u	upper bounds for parameters
nbd	indicates which bounds are present
task	if an error occurs, contains a human-readable error message
info	=0 on success; =-6 if nbd(k) was invalid; =-7 if both limits are given but l(k) $>$ u(k)
k	index of last errournous parameter

Definition at line 16 of file errclb.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.8 src/formk.f File Reference

Functions/Subroutines

- subroutine formk (n, nsub, ind, nenter, ileave, indx2, iupdat, updatd, wn, wn1, m, ws, wy, sy, theta, col, head, info)

  Forms the LEL^T factorization of the indefinite matrix K.

12.8.1 Function/Subroutine Documentation

12.8.1.1 formk()

```
subroutine formk (
        integer n,
        integer nsub,
        integer, dimension(n) ind,
        integer nenter,
        integer ileave,
        integer, dimension(n) indx2,
        integer iupdat,
        logical updatd,
        double precision, dimension(2*m, 2*m) wn,
```

```
double precision, dimension(2*m, 2*m) wn1,
integer m,
double precision, dimension(n, m) ws,
double precision, dimension(n, m) wy,
double precision, dimension(m, m) sy,
double precision theta,
integer col,
integer head,
integer info )
```

This subroutine forms the LEL^T factorization of the indefinite matrix

K = [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta∗S'AA'S] where E = [-I 0] [0 I]

The matrix K can be shown to be equal to the matrix M^[-1]N occurring in section 5.1 of [1], as well as to the matrix Mbar^[-1] Nbar in section 5.3.

Parameters

n	On entry n is the dimension of the problem. On exit n is unchanged.
nsub	On entry nsub is the number of subspace variables in free set. On exit nsub is not changed.
ind	On entry ind specifies the indices of subspace variables. On exit ind is unchanged.
nenter	On entry nenter is the number of variables entering the free set. On exit nenter is unchanged.
ileave	On entry indx2(ileave),...,indx2(n) are the variables leaving the free set. On exit ileave is unchanged.
indx2	On entry indx2(1),...,indx2(nenter) are the variables entering the free set, while indx2(ileave),...,indx2(n) are the variables leaving the free set. On exit indx2 is unchanged.
iupdat	On entry iupdat is the total number of BFGS updates made so far. On exit iupdat is unchanged.
updatd	On entry 'updatd' is true if the L-BFGS matrix is updated. On exit 'updatd' is unchanged.
wn	On entry wn is unspecified. On exit the upper triangle of wn stores the LEL^T factorization of the 2∗col x 2∗col indefinite matrix [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta∗S'AA'S]
wn1	On entry wn1 stores the lower triangular part of [Y' ZZ'Y L_a'+R_z'] [L_a+R_z S'AA'S] in the previous iteration. On exit wn1 stores the corresponding updated matrices. The purpose of wn1 is just to store these inner products so they can be easily updated and inserted into wn.
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
sy	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.

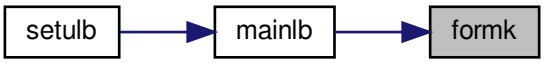
Parameters

col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
head	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
info	On entry info is unspecified. On exit info • = 0 for normal return; • = -1 when the 1st Cholesky factorization failed; • = -2 when the 2st Cholesky factorization failed.

Definition at line 89 of file formk.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.9 src/formt.f File Reference

Functions/Subroutines

- subroutine formt (m, wt, sy, ss, col, theta, info)

  Forms the upper half of the pos. def. and symm. T.

12.9.1 Function/Subroutine Documentation

12.9.1.1 formt()

```
subroutine formt (
        integer m,
        double precision, dimension(m, m) wt,
        double precision, dimension(m, m) sy,
        double precision, dimension(m, m) ss,
        integer col,
        double precision theta,
        integer info )
```

This subroutine forms the upper half of the pos. def. and symm. T = theta∗SS + L∗D^(-1)∗L', stores T in the upper triangle of the array wt, and performs the Cholesky factorization of T to produce J∗J', with J' stored in the upper triangle of wt.

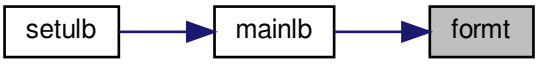
Parameters

m	history size of approximated Hessian
wt	part of L-BFGS matrix
sy	part of L-BFGS matrix
ss	part of L-BFGS matrix
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
info	error/success indicator

Definition at line 21 of file formt.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.10 src/freev.f File Reference

Functions/Subroutines

- subroutine freev (n, nfree, index, nenter, ileave, indx2, iwhere, wrk, updatd, cnstnd, iprint, iter)

  This subroutine counts the entering and leaving variables when iter $>$ 0, and finds the index set of free and active variables at the GCP.

12.10.1 Function/Subroutine Documentation

12.10.1.1 freev()

```
subroutine freev (
    integer n,
    integer nfree,
    integer, dimension(n) index,
    integer nenter,
    integer ileave,
    integer, dimension(n) indx2,
    integer, dimension(n) iwhere,
    logical wrk,
    logical updatd,
    logical cnstnd,
    integer iprint,
    integer iter )
```

This subroutine counts the entering and leaving variables when iter $>$ 0, and finds the index set of free and active variables at the GCP.

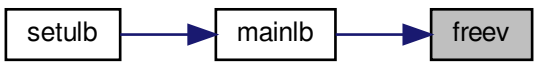
Parameters

n	number of parameters
nfree	number of free parameters, i.e., those not at their bounds
index	for i=1,...,nfree, index(i) are the indices of free variables for i=nfree+1,...,n, index(i) are the indices of bound variables On entry after the first iteration, index gives the free variables at the previous iteration. On exit it gives the free variables based on the determination in cauchy using the array iwhere.
nenter	On exit nenter is the number of variables that entered the free set this iteration (were active, now free at the GCP).
ileave	On exit indx2(ileave),...,indx2(n) list the variables that left the free set this iteration. ileave starts at n+1 and is decremented each time a leaving variable is recorded.
indx2	On entry indx2 is unspecified. On exit with iter>0, indx2 indicates which variables have changed status since the previous iteration. For i= 1,...,nenter, indx2(i) have changed from bound to free. For i= ileave+1,...,n, indx2(i) have changed from free to bound.
iwhere	On entry iwhere(i) classifies each variable's bound status (set by cauchy): <=0 means free at GCP, >0 means at-bound. Used here to compare against the previous index/nfree to detect leaving and entering variables.
wrk	On exit .true. if the active-set or L-BFGS bookkeeping has changed enough that the workspace WN needs to be rebuilt (some variable entered/left, or updatd is .true.).
updatd	On entry .true. if the L-BFGS matrix was updated in the previous iteration. Combined with the entering/leaving counts to set wrk.
cnstnd	Whether bounds are present (true if at least one variable is bounded). When false, the entering/leaving counting loop is skipped.
iprint	Console output flag (>=99 prints summary, >=100 prints per-variable change records).
iter	Current outer iteration number. The entering/leaving counting loop only runs when iter > 0 (the first iteration has no "previous" set to compare against).

Definition at line 47 of file freev.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.11 src/hpsolb.f File Reference

Functions/Subroutines

- subroutine hpsolb (n, t, iorder, iheap)

  This subroutine sorts out the least element of t, and puts the remaining elements of t in a heap.

12.11.1 Function/Subroutine Documentation

12.11.1.1 hpsolb()

```
subroutine hpsolb (
    integer n,
    double precision, dimension(n) t,
    integer, dimension(n) iorder,
    integer iheap )
```

Parameters

n	On entry n is the dimension of the arrays t and iorder. On exit n is unchanged.
t	On entry t stores the elements to be sorted. On exit t(n) stores the least elements of t, and t(1) to t(n-1) stores the remaining elements in the form of a heap.
iorder	On entry iorder(i) is the index of t(i). On exit iorder(i) is still the index of t(i), but iorder may be permuted in accordance with t.
iheap	On entry iheap should be set as follows: • iheap .eq. 0 if t(1) to t(n) is not in the form of a heap, • iheap .ne. 0 if otherwise. On exit iheap is unchanged.

Definition at line 21 of file hpsolb.f.

Referenced by cauchy().

Here is the caller graph for this function:

12.12 src/lnsrlb.f File Reference

Functions/Subroutines

- subroutine lnsrlb (n, l, u, nbd, x, f, fold, gd, gdold, g, d, r, t, z, stp, dnorm, dtd, xstep, stpmx, iter, ifun, iback, nfgv, info, task, boxed, cnstnd, csave, isave, dsave)

  This subroutine calls subroutine dcsrch from the Minpack2 library to perform the line search. Subroutine dscrch is safeguarded so that all trial points lie within the feasible region.

12.12.1 Function/Subroutine Documentation

12.12.1.1 lnsrlb() subroutine lnsrlb (

```
integer n,
double precision, dimension(n) l,
double precision, dimension(n) u,
integer, dimension(n) nbd,
double precision, dimension(n) x,
double precision f,
double precision fold,
double precision gd,
double precision gdold,
double precision, dimension(n) g,
double precision, dimension(n) d,
double precision, dimension(n) r,
double precision, dimension(n) t,
double precision, dimension(n) z,
double precision stp,
double precision dnorm,
double precision dtd,
double precision xstep,
double precision stpmx,
integer iter,
integer ifun,
integer iback,
integer nfgv,
integer info,
character*60 task,
logical boxed,
logical cnstnd,
character*60 csave,
integer, dimension(2) isave,
double precision, dimension(13) dsave )
```

Parameters

n	number of parameters
l	lower bounds of parameters
u	upper bounds of parameters
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, and • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
x	position
f	function value at x
fold	Function value at the start of this line search (i.e. the accepted value from the previous iteration). Saved at entry so the caller can restore x, g, f if the line search fails.

Parameters

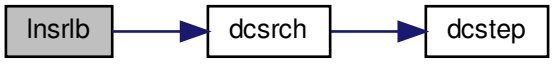
gd	Directional derivative g'd at the current trial step. Computed on every entry and passed to dcsrch as its g argument.
gdold	Directional derivative at stp=0 (i.e. the initial g'd before any line-search progress this iteration). Saved on the first call and used by mainlb to test the curvature condition after the line search returns.
g	Gradient of f at x.
d	Search direction (z - x_current). Length-n vector; the candidate step is t + stp∗d.
r	Workspace: copy of g at the start of this line search. Used alongside fold to restore the previous iterate on abnormal line-search termination (mainlb does the restore from r and t).
t	Workspace: copy of x at the start of this line search.
z	Pre-projected candidate (cauchy/subsm output). When stp=1 exactly, lnsrlb sets x := z directly; otherwise it computes x := t + stp∗d.
stp	Current trial step length. On the first entry of a line search lnsrlb initialises stp; subsequent dcsrch calls update it.
dnorm	2-norm of d (||d||).
dtd	Squared 2-norm of d (d'd).
xstep	On exit stp ∗ ||d||, the actual length of the step in x-space.
stpmx	Maximum allowed step. For unconstrained problems set to a large constant (1e10); for bounded problems lnsrlb scans the active bounds and tightens stpmx so x + stpmx∗d stays feasible.
iter	Outer iteration number from mainlb.
ifun	On exit number of f/g evaluations performed in this line search; reset to 0 on each new line search.
iback	On exit number of "backtracks" (ifun - 1). mainlb aborts the line search if iback >= 20.
nfgv	Cumulative count of f/g evaluations across all iterations; incremented by 1 per evaluation requested.
info	On exit 0 on success; -4 if the projected directional derivative gd is non-negative on the first call (no descent possible).
task	Reverse-comm task. Initial entry: 'START'. While the line search is running, lnsrlb returns 'FG_LNSRCH' (the user evaluates f, g at the new x and re-enters with task starting with 'FG_LN'). On line-search success lnsrlb returns 'NEW_X'.
boxed	.true. if every variable has both lower and upper bounds. When true, the initial trial step is unit (stp=1) regardless of d's magnitude.
cnstnd	.true. if the problem has at least one bound. Controls the stpmx-from-bounds scan.
csave	working array
isave	working array
dsave	working array

Definition at line 70 of file lnsrlb.f.

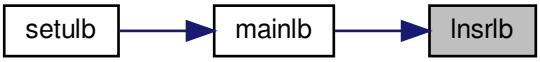
References dcsrch().

Referenced by mainlb().

Here is the call graph for this function:



Here is the caller graph for this function:



12.13 src/mainlb.f File Reference

Functions/Subroutines

- subroutine mainlb (n, m, x, l, u, nbd, f, g, factr, pgtol, ws, wy, sy, ss, wt, wn, snd, z, r, d, t, xp, wa, index, iwhere, indx2, task, iprint, csave, lsave, isave, dsave)

  This subroutine solves bound constrained optimization problems by using the compact formula of the limited memory BFGS updates.

12.13.1 Function/Subroutine Documentation

12.13.1.1 mainlb()

```
subroutine mainlb (
    integer n,
    integer m,
    double precision, dimension(n) x,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    integer, dimension(n) nbd,
    double precision f,
    double precision, dimension(n) g,
    double precision factr,
    double precision pgtol,
    double precision, dimension(n, m) ws,
    double precision, dimension(n, m) wy,
    double precision, dimension(m, m) sy,
    double precision, dimension(m, m) ss,
    double precision, dimension(m, m) wt,
    double precision, dimension(2*m, 2*m) wn,
    double precision, dimension(2*m, 2*m) snd,
    double precision, dimension(n) z,
    double precision, dimension(n) r,
    double precision, dimension(n) d,
    double precision, dimension(n) t,
    double precision, dimension(n) xp,
    double precision, dimension(8*m) wa,
    integer, dimension(n) index,
    integer, dimension(n) iwhere,
    integer, dimension(n) indx2,
    character*60 task,
```

```
integer iprint,
character*60 csave,
logical, dimension(4) lsave,
integer, dimension(23) isave,
double precision, dimension(29) dsave )
```

This subroutine solves bound constrained optimization problems by using the compact formula of the limited memory BFGS updates.

Parameters

n	On entry n is the number of variables. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit m is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
f	On first entry f is unspecified. On final exit f is the value of the function at x.
g	On first entry g is unspecified. On final exit g is the value of the gradient at x.
factr	On entry factr >= 0 is specified by the user. The iteration will stop when (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr∗epsmch where epsmch is the machine precision, which is automatically generated by the code. On exit factr is unchanged.
pgtol	On entry pgtol >= 0 is specified by the user. The iteration will stop when max{|proj g_i | i = 1, ..., n} <= pgtol where pg_i is the ith component of the projected gradient. On exit pgtol is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
sy	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.
ss	On entry this stores S'S, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wt	On entry this stores the Cholesky factorization of (theta∗S'S+LD^(-1)L'), that defines the limited memory BFGS matrix. See eq. (2.26) in [3]. On exit this array is unchanged.

Parameters

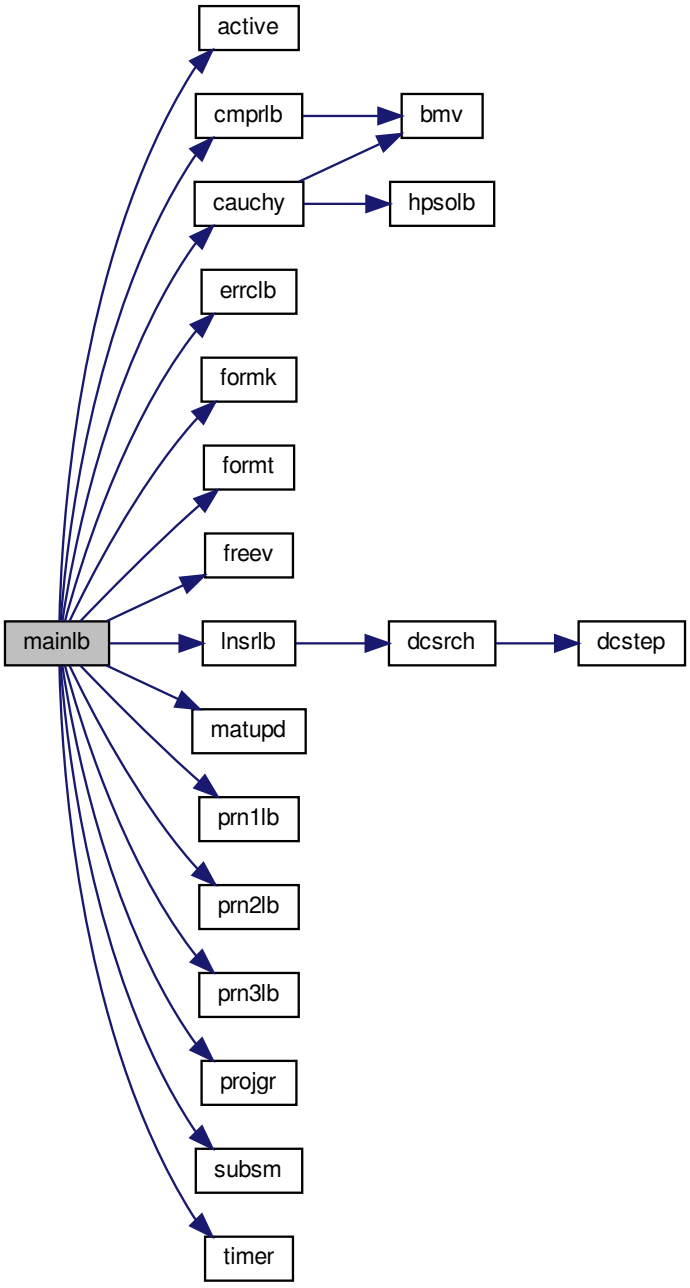
wn	working array used to store the LEL^T factorization of the indefinite matrix K = [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta∗S'AA'S] where E = [-I 0] [0 I]
snd	working array used to store the lower triangular part of N = [Y' ZZ'Y L_a'+R_z'] [L_a +R_z S'AA'S]
z	working array used at different times to store the Cauchy point and the Newton point.
r	working array
d	working array
t	working array
xp	working array used to safeguard the projected Newton direction
wa	working array
index	In subroutine freev, index is used to store the free and fixed variables at the Generalized Cauchy Point (GCP).
iwhere	working array used to record the status of the vector x for GCP computation. iwhere(i)= • 0 or -3 if x(i) is free and has bounds, • 1 if x(i) is fixed at l(i), and l(i) .ne. u(i) • 2 if x(i) is fixed at u(i), and u(i) .ne. l(i) • 3 if x(i) is always fixed, i.e., u(i)=x(i)=l(i) • -1 if x(i) is always free, i.e., no bounds on it.
indx2	working array Within subroutine cauchy, indx2 corresponds to the array iorder. In subroutine freev, a list of variables entering and leaving the free set is stored in indx2, and it is passed on to subroutine formk with this information.
task	working string indicating the current job when entering and leaving this subroutine.
iprint	It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
csave	working string
lsave	working array
isave	working array
dsave	working array

Definition at line 128 of file mainlb.f.

References active(), cauchy(), cmprlb(), errclb(), formk(), formt(), freev(), lnsrlb(), matupd(), prn1lb(), prn2lb(), prn3lb(), projgr(), subsm(), and timer().

Referenced by setulb().

Here is the call graph for this function:



Here is the caller graph for this function:

12.14 src/matupd.f File Reference

Functions/Subroutines

- subroutine matupd (n, m, ws, wy, sy, ss, d, r, itail, iupdat, col, head, theta, rr, dr, stp, dtd)

  This subroutine updates matrices WS and WY, and forms the middle matrix in B.

12.14.1 Function/Subroutine Documentation

12.14.1.1 matupd() subroutine matupd (

```
integer n,
integer m,
double precision, dimension(n, m) ws,
double precision, dimension(n, m) wy,
double precision, dimension(m, m) sy,
double precision, dimension(m, m) ss,
double precision, dimension(n) d,
double precision, dimension(n) r,
integer itail,
integer iupdat,
integer col,
integer head,
double precision theta,
double precision rr,
double precision dr,
double precision stp,
double precision dtd )
```

This subroutine updates matrices WS and WY, and forms the middle matrix in B.

Parameters

n	On entry n is the number of variables. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit m is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.

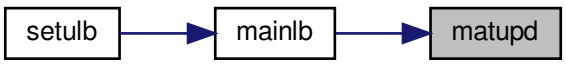
Parameters

wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
sy	On entry this stores S'Y, that defines the limited memory BFGS matrix. On exit this array is unchanged.
ss	On entry this stores S'S, that defines the limited memory BFGS matrix. On exit this array is unchanged.
d	Search direction at the current iteration. After the line search, the new s-vector is stp∗d; matupd stores it as a column of WS (writing d directly, since the stp scaling is folded into the stored ss diagonal entry below).
r	The accepted gradient difference y = g_{k+1} - g_k. Stored as a column of WY.
itail	On entry the column index in WS/WY that the previous update wrote. On exit the column index this update wrote; advances cyclically modulo m.
iupdat	Total number of L-BFGS updates performed so far (incremented by mainlb before each matupd call). When iupdat <= m, col simply grows; when iupdat > m, the history wraps and the oldest column is discarded.
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
head	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
rr	Squared 2-norm of r (i.e. ||y||^2 = y'y). Used to set theta := rr/dr = y'y / (s'y), the standard initial Hessian scaling for the next L-BFGS update.
dr	Inner product d'r = s'y / stp (the curvature condition; must be positive for the update to be safely accepted – mainlb checks this and skips matupd if dr <= eps∗rr).
stp	Line-search step length. Used to recover the s-vector from d (s = stp∗d) when computing ss(col,col) = stp^2 ∗ d'd.
dtd	Squared 2-norm of d (i.e. d'd). Combined with stp to form ss(col,col) = stp^2 ∗ dtd = ||s||^2 for the new column.

Definition at line 64 of file matupd.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.15 src/prn1lb.f File Reference

Functions/Subroutines

- subroutine prn1lb (n, m, l, u, x, iprint, itfile, epsmch)

  This subroutine prints the input data, initial point, upper and lower bounds of each variable, machine precision, as well as the headings of the output.

12.15.1 Function/Subroutine Documentation

12.15.1.1 prn1lb()

```
subroutine prn1lb (
    integer n,
    integer m,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    double precision, dimension(n) x,
    integer iprint,
    integer itfile,
    double precision epsmch )
```

This subroutine prints the input data, initial point, upper and lower bounds of each variable, machine precision, as well as the headings of the output.

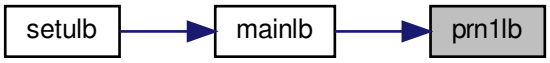
Parameters

n	On entry n is the number of variables. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections allowed in the limited memory matrix. On exit m is unchanged.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
iprint	It controls the frequency and type of output generated: • iprint$<$0 no output is generated; • iprint=0 print only one line at the last iteration; • 0$<$iprint$<$99 print also f and $\|$proj g$\|$ every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint$>$100 print details of every iteration including x and g; When iprint $>$ 0, the file iterate.dat will be created to summarize the iteration.
itfile	unit number of iterate.dat file
epsmch	machine precision epsilon

Definition at line 39 of file prn1lb.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.16 src/prn2lb.f File Reference

Functions/Subroutines

- subroutine prn2lb (n, x, f, g, iprint, itfile, iter, nfgv, nact, sbgnrm, nseg, word, iword, iback, stp, xstep)

  This subroutine prints out new information after a successful line search.

12.16.1 Function/Subroutine Documentation

12.16.1.1 prn2lb()

```
subroutine prn2lb (
    integer n,
    double precision, dimension(n) x,
    double precision f,
    double precision, dimension(n) g,
    integer iprint,
    integer itfile,
    integer iter,
    integer nfgv,
    integer nact,
    double precision sbgnrm,
    integer nseg,
    character*3 word,
    integer iword,
    integer iback,
    double precision stp,
    double precision xstep )
```

This subroutine prints out new information after a successful line search.

Parameters

n	On entry n is the number of variables. On exit n is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
f	On first entry f is unspecified. On final exit f is the value of the function at x.

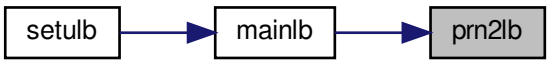
Parameters

g	On first entry g is unspecified. On final exit g is the value of the gradient at x.
iprint	It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
itfile	unit number of iterate.dat file
iter	Current outer iteration number; printed on the per-iterate summary line.
nfgv	Cumulative number of f/g evaluations across the whole run; logged into the iterate.dat record.
nact	Number of variables active at their bounds at the current iterate.
sbgnrm	Infinity norm of the projected gradient at x; the natural first-order optimality measure printed alongside f.
nseg	Number of breakpoint segments traversed by cauchy at the current iteration; printed for diagnostic accounting.
word	On exit a 3-character status code summarising the subspace solution: 'con' (converged), 'bnd' (hit a bound), 'TNT' (truncated-Newton step used), or '—' otherwise. Determined from iword.
iword	Status code from subsm: 0 = subspace minimisation converged, 1 = stopped at a bound, 5 = truncated-Newton step used. Maps to word.
iback	Number of backtracks the line search performed.
stp	Final step length accepted by the line search.
xstep	stp $*$ ||d|| – the actual length of the step in x-space.

Definition at line 53 of file prn2lb.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.17 src/prn3lb.f File Reference

Functions/Subroutines

- subroutine prn3lb (n, x, f, task, iprint, info, itfile, iter, nfgv, nintol, nskip, nact, sbgnrm, time, nseg, word, iback, stp, xstep, k, cachyt, sbtime, lnscht)

This subroutine prints out information when either a built-in convergence test is satisfied or when an error message is generated.

12.17.1 Function/Subroutine Documentation

12.17.1.1 prn3lb()

```
subroutine prn3lb (
    integer n,
    double precision, dimension(n) x,
    double precision f,
    character*60 task,
    integer iprint,
    integer info,
    integer itfile,
    integer iter,
    integer nfgv,
    integer nintol,
    integer nskip,
    integer nact,
    double precision sbgnrm,
    double precision time,
    integer nseg,
    character*3 word,
    integer iback,
    double precision stp,
    double precision xstep,
    integer k,
    double precision cachyt,
    double precision sbtime,
    double precision lnscht )
```

This subroutine prints out information when either a built-in convergence test is satisfied or when an error message is generated.

Parameters

n	On entry n is the number of variables. On exit n is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
f	On first entry f is unspecified. On final exit f is the value of the function at x.
task	working string indicating the current job when entering and leaving this subroutine.
iprint	Console output flag; same convention as in setulb. Negative suppresses all output. >=0 prints the per-run summary. >=1 also writes to iterate.dat. >=100 also prints the final x vector.
info	Termination status. 0 on normal convergence. Negative values map to specific error messages: -1, -2 (Cholesky not pos.def. in formk's 1st/2nd factor), -3 (formt's Cholesky), -4 (non-descent direction caught by lnsrlb), -5 (>10 evals in a line search), -6 (invalid nbd(k)), -7 (l(k) > u(k)), -8 (singular triangular system), -9 (>20 evals in the last line search).
itfile	unit number of iterate.dat file
iter	Total number of outer iterations.
nfgv	Total number of f/g evaluations across the run.
nintol	Total number of breakpoint segments traversed by cauchy.

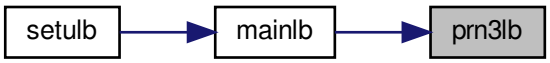
Parameters

nskip	Number of L-BFGS updates that were skipped because the curvature condition failed (s'y too small).
nact	Number of active bounds at the final generalised Cauchy point.
sbgnrm	Final infinity norm of the projected gradient at x.
time	Total wall-clock time of the optimisation run, in seconds.
nseg	Number of breakpoint segments traversed at the LAST cauchy call (used in the iterate.dat record on abnormal exits).
word	3-character status code from prn2lb: 'con', 'bnd', 'TNT', or '—' (see prn2lb.f). Logged in iterate.dat on info=-4 or info=-9 termination.
iback	Number of backtracks in the LAST line search.
stp	Final step length from the LAST line search.
xstep	stp $*$ ||d|| from the LAST line search (i.e. the actual step length in x-space).
k	Index of the offending parameter when info is -6 (invalid nbd) or -7 (infeasible bound). Set by errclb.
cachyt	Cumulative wall-clock time spent in cauchy, in seconds.
sbtime	Cumulative wall-clock time spent in subsm, in seconds.
lnscht	Cumulative wall-clock time spent in lnsrlb, in seconds.

Definition at line 56 of file prn3lb.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.18 src/projgr.f File Reference

Functions/Subroutines

- subroutine projgr (n, l, u, nbd, x, g, sbgnrm)

  This subroutine computes the infinity norm of the projected gradient.

12.18.1 Function/Subroutine Documentation

12.18.1.1 projgr()

```
subroutine projgr (
    integer n,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    integer, dimension(n) nbd,
    double precision, dimension(n) x,
    double precision, dimension(n) g,
    double precision sbgnrm )
```

This subroutine computes the infinity norm of the projected gradient.

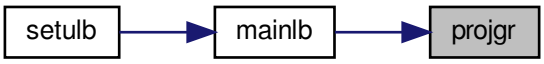
Parameters

n	On entry n is the number of variables. On exit n is unchanged.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
x	On entry x is an approximation to the solution. On exit x is unchanged.
g	On entry g is the gradient. On exit g is unchanged.
sbgnrm	infinity norm of projected gradient

Definition at line 33 of file projgr.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.19 src/setulb.f File Reference

Functions/Subroutines

- subroutine setulb (n, m, x, l, u, nbd, f, g, factr, pgtol, wa, iwa, task, iprint, csave, lsave, isave, dsave)

  This subroutine partitions the working arrays wa and iwa, and then uses the limited memory BFGS method to solve the bound constrained optimization problem by calling mainlb.

12.19.1 Function/Subroutine Documentation

12.19.1.1 setulb()

```
subroutine setulb (
    integer n,
    integer m,
    double precision, dimension(n) x,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    integer, dimension(n) nbd,
    double precision f,
    double precision, dimension(n) g,
    double precision factr,
    double precision pgtol,
    double precision, dimension(2*m*n + 5*n + 11*m*m + 8*m) wa,
    integer, dimension(3*n) iwa,
    character*60 task,
    integer iprint,
    character*60 csave,
    logical, dimension(4) lsave,
    integer, dimension(44) isave,
    double precision, dimension(29) dsave )
```

This subroutine partitions the working arrays wa and iwa, and then uses the limited memory BFGS method to solve the bound constrained optimization problem by calling mainlb. (The direct method will be used in the subspace minimization.)

Parameters

n	On entry n is the dimension of the problem. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
x	On entry x is an approximation to the solution. On exit x is the current approximation.
l	On entry l is the lower bound on x. On exit l is unchanged.
u	On entry u is the upper bound on x. On exit u is unchanged.
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, and • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
f	On first entry f is unspecified. On final exit f is the value of the function at x.
g	On first entry g is unspecified. On final exit g is the value of the gradient at x.

Parameters

factr	On entry factr >= 0 is specified by the user. The iteration will stop when (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr∗epsmch where epsmch is the machine precision, which is automatically generated by the code. Typical values for factr: • 1.d+12 for low accuracy; • 1.d+7 for moderate accuracy; • 1.d+1 for extremely high accuracy. On exit factr is unchanged.
pgtol	On entry pgtol >= 0 is specified by the user. The iteration will stop when max{|proj g_i | i = 1, ..., n} <= pgtol where pg_i is the ith component of the projected gradient. On exit pgtol is unchanged.
wa	working array
iwa	working array
task	working string indicating the current job when entering and quitting this subroutine.
iprint	Must be set by the user. It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.
csave	working string
lsave	working array; On exit with 'task' = NEW_X, the following information is available: • If lsave(1) = .true. then the initial X has been replaced by its projection in the feasible set; • If lsave(2) = .true. then the problem is constrained; • If lsave(3) = .true. then each variable has upper and lower bounds;

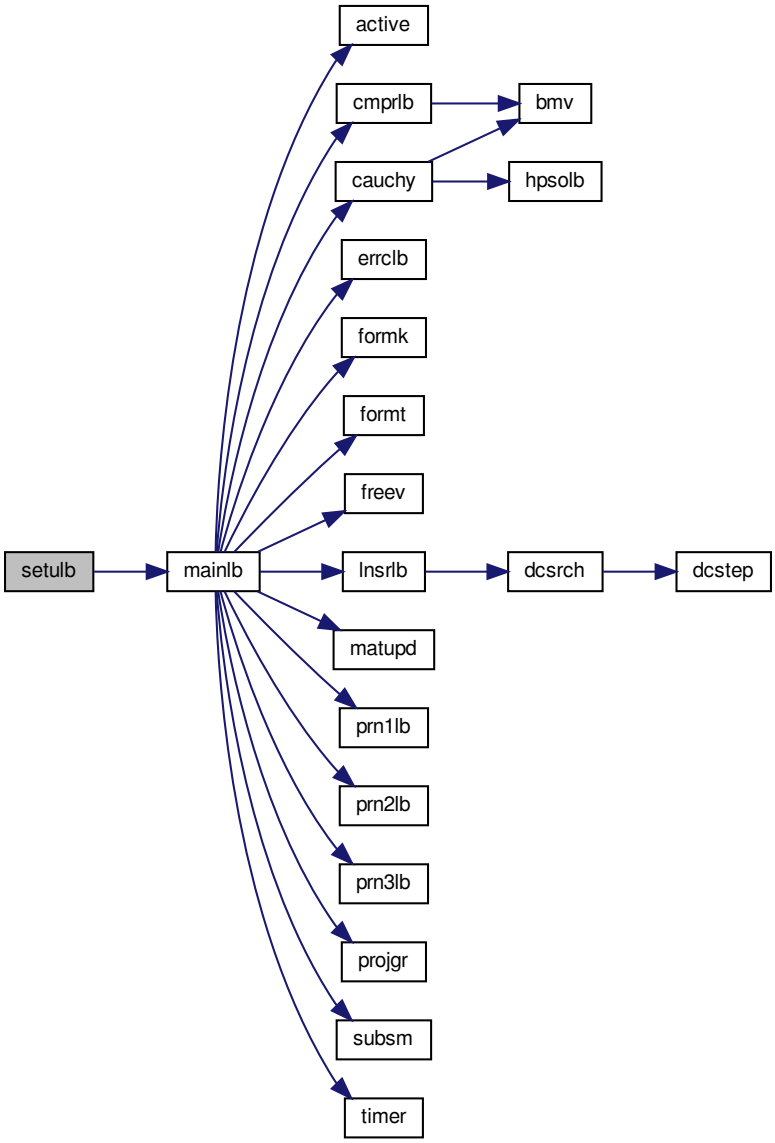
Parameters

isave	working array; On exit with 'task' = NEW_X, the following information is available: • isave(22) = the total number of intervals explored in the search of Cauchy points; • isave(26) = the total number of skipped BFGS updates before the current iteration; • isave(30) = the number of current iteration; • isave(31) = the total number of BFGS updates prior the current iteration; • isave(33) = the number of intervals explored in the search of Cauchy point in the current iteration; • isave(34) = the total number of function and gradient evaluations; • isave(36) = the number of function value or gradient evaluations in the current iteration; • if isave(37) = 0 then the subspace argmin is within the box; • if isave(37) = 1 then the subspace argmin is beyond the box; • isave(38) = the number of free variables in the current iteration; • isave(39) = the number of active constraints in the current iteration; • n + 1 - isave(40) = the number of variables leaving the set of active constraints in the current iteration; • isave(41) = the number of variables entering the set of active constraints in the current iteration.
dsave	working array; On exit with 'task' = NEW_X, the following information is available: • dsave(1) = current 'theta' in the BFGS matrix; • dsave(2) = f(x) in the previous iteration; • dsave(3) = factr∗epsmch; • dsave(4) = 2-norm of the line search direction vector; • dsave(5) = the machine precision epsmch generated by the code; • dsave(7) = the accumulated time spent on searching for Cauchy points; • dsave(8) = the accumulated time spent on subspace minimization; • dsave(9) = the accumulated time spent on line search; • dsave(11) = the slope of the line search function at the current point of line search; • dsave(12) = the maximum relative step length imposed in line search; • dsave(13) = the infinity norm of the projected gradient; • dsave(14) = the relative step length in the line search; • dsave(15) = the slope of the line search function at the starting point of the line search; • dsave(16) = the square of the 2-norm of the line search direction vector.

Definition at line 188 of file setulb.f.

References mainlb().

Here is the call graph for this function:



12.20 src/subsm.f File Reference

Functions/Subroutines

- subroutine subsm (n, m, nsub, ind, l, u, nbd, x, d, xp, ws, wy, theta, xx, gg, col, head, iword, wv, wn, iprint)

  Performs the subspace minimization.

12.20.1 Function/Subroutine Documentation

12.20.1.1 subsm()

```
subroutine subsm (
    integer n,
    integer m,
    integer nsub,
    integer, dimension(nsub) ind,
    double precision, dimension(n) l,
    double precision, dimension(n) u,
    integer, dimension(n) nbd,
    double precision, dimension(n) x,
    double precision, dimension(n) d,
    double precision, dimension(n) xp,
    double precision, dimension(n, m) ws,
    double precision, dimension(n, m) wy,
    double precision theta,
    double precision, dimension(n) xx,
    double precision, dimension(n) gg,
    integer col,
    integer head,
    integer iword,
    double precision, dimension(2*m) wv,
    double precision, dimension(2*m, 2*m) wn,
    integer iprint )
```

Given xcp, l, u, r, an index set that specifies the active set at xcp, and an l-BFGS matrix B (in terms of WY, WS, SY, WT, head, col, and theta), this subroutine computes an approximate solution of the subspace problem

(P) min Q(x) = r'(x-xcp) + 1/2 (x-xcp)' B (x-xcp)

subject to l<=x<=u x_i=xcp_i for all i in A(xcp)

along the subspace unconstrained Newton direction

d = -(Z'BZ)^(-1) r.

The formula for the Newton direction, given the L-BFGS matrix and the Sherman-Morrison formula, is

d = (1/theta)r + (1/theta*2) Z'WK^(-1)W'Z r.

where K = [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta*S'AA'S]

Note that this procedure for computing d differs from that described in [1]. One can show that the matrix K is equal to the matrix M^[-1]N in that paper.

Parameters

n	On entry n is the dimension of the problem. On exit n is unchanged.
m	On entry m is the maximum number of variable metric corrections used to define the limited memory matrix. On exit m is unchanged.
nsub	On entry nsub is the number of free variables. On exit nsub is unchanged.
ind	On entry ind specifies the coordinate indices of free variables. On exit ind is unchanged.
l	On entry l is the lower bound of x. On exit l is unchanged.
u	On entry u is the upper bound of x. On exit u is unchanged.

Parameters

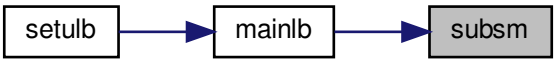
nbd	On entry nbd represents the type of bounds imposed on the variables, and must be specified as follows: nbd(i)= • 0 if x(i) is unbounded, • 1 if x(i) has only a lower bound, • 2 if x(i) has both lower and upper bounds, and • 3 if x(i) has only an upper bound. On exit nbd is unchanged.
x	On entry x specifies the Cauchy point xcp. On exit x(i) is the minimizer of Q over the subspace of free variables.
d	On entry d is the reduced gradient of Q at xcp. On exit d is the Newton direction of Q.
xp	used to safeguard the projected Newton direction
xx	On entry it holds the current iterate. On output it is unchanged.
gg	On entry it holds the gradient at the current iterate. On output it is unchanged.
ws	On entry this stores S, a set of s-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
wy	On entry this stores Y, a set of y-vectors, that defines the limited memory BFGS matrix. On exit this array is unchanged.
theta	On entry theta is the scaling factor specifying B_0 = theta I. On exit theta is unchanged.
col	On entry col is the actual number of variable metric corrections stored so far. On exit col is unchanged.
head	On entry head is the location of the first s-vector (or y-vector) in S (or Y). On exit col is unchanged.
iword	On entry iword is unspecified. On exit iword specifies the status of the subspace solution. iword = • 0 if the solution is in the box, • 1 if some bound is encountered.
wv	working array
wn	On entry the upper triangle of wn stores the LEL^T factorization of the indefinite matrix K = [-D -Y'ZZ'Y/theta L_a'-R_z'] [L_a -R_z theta∗S'AA'S] where E = [-I 0] [0 I] On exit wn is unchanged.
iprint	must be set by the user; It controls the frequency and type of output generated: • iprint<0 no output is generated; • iprint=0 print only one line at the last iteration; • 0<iprint<99 print also f and |proj g| every iprint iterations; • iprint=99 print details of every iteration except n-vectors; • iprint=100 print also the changes of active set and final x; • iprint>100 print details of every iteration including x and g; When iprint > 0, the file iterate.dat will be created to summarize the iteration.

Historical note: this routine used to take an `info` output parameter for an "ill-conditioned K" status. Since K's conditioning is checked in `formt/formk` (which fail loudly with their own `info`) and the two `dtrsm` calls inside `subsm` cannot fail on a non-singular factor, the parameter was always 0 on exit and has been removed.

Definition at line 126 of file subsm.f.

Referenced by mainlb().

Here is the caller graph for this function:



12.21 src/timer.f File Reference

Functions/Subroutines

- subroutine timer (ttime)

  This routine computes cpu time in double precision.

12.21.1 Function/Subroutine Documentation

12.21.1.1 timer() `subroutine timer ( double precision ttime )`

This routine computes cpu time in double precision; it makes use of the intrinsic f90 cpu_time therefore a conversion type is needed.

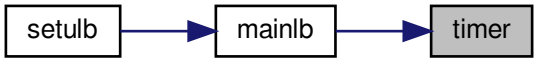
Parameters

ttime	CPU time in double precision

Definition at line 10 of file timer.f.

Referenced by mainlb().

Here is the caller graph for this function:



13 Example Documentation

13.1 driver1.f

This simple driver demonstrates how to call the L-BFGS-B code to solve a sample problem (the extended Rosenbrock function subject to bounds on the variables). The dimension n of this problem is variable. (Fortran-77 version)

```
1 c> \file driver1.f
2
3 c
4 c  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c  or "3-clause license")
6 c  Please read attached file License.txt
7 c
8 c                           DRIVER 1 in Fortran 77
9 c     ----------------------------------------------------------------
10 c               SIMPLE DRIVER FOR L-BFGS-B (version 3.0)
11 c     ----------------------------------------------------------------
12 c
13 c        L-BFGS-B is a code for solving large nonlinear optimization
14 c             problems with simple bounds on the variables.
15 c
16 c        The code can also be used for unconstrained problems and is
17 c        as efficient for these problems as the earlier limited memory
18 c                         code L-BFGS.
19 c
20 c        This is the simplest driver in the package. It uses all the
21 c                    default settings of the code.
22 c
23 c
24 c     References:
25 c
26 c        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 c        memory algorithm for bound constrained optimization",
28 c        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 c
30 c        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 c        Subroutines for Large Scale Bound Constrained Optimization"
32 c        Tech. Report, NAM-11, EECS Department, Northwestern University,
33 c        1994.
34 c
35 c
36 c          (Postscript files of these papers are available via anonymous
37 c           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 c
39 c                               *  *  *
40 c
41 c         March 2011   (latest revision)
42 c         Optimization Center at Northwestern University
43 c         Instituto Tecnologico Autonomo de Mexico
44 c
45 c         Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
46 c         L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
47 c         Optimization"  (2011). To appear in  ACM Transactions on
48 c         Mathematical Software,
49
50 c     ----------------------------------------------------------------
51 c             DESCRIPTION OF THE VARIABLES IN L-BFGS-B
52 c     ----------------------------------------------------------------
53 c
54 c     n is an INTEGER variable that must be set by the user to the
55 c       number of variables.  It is not altered by the routine.
```

```
c
c     m is an INTEGER variable that must be set by the user to the
c       number of corrections used in the limited memory matrix.
c       It is not altered by the routine.  Values of m < 3  are
c       not recommended, and large values of m can result in excessive
c       computing time. The range  3 <= m <= 20 is recommended.
c
c     x is a DOUBLE PRECISION array of length n.  On initial entry
c       it must be set by the user to the values of the initial
c       estimate of the solution vector.  Upon successful exit, it
c       contains the values of the variables at the best point
c       found (usually an approximate solution).
c
c     l is a DOUBLE PRECISION array of length n that must be set by
c       the user to the values of the lower bounds on the variables. If
c       the i-th variable has no lower bound, l(i) need not be defined.
c
c     u is a DOUBLE PRECISION array of length n that must be set by
c       the user to the values of the upper bounds on the variables. If
c       the i-th variable has no upper bound, u(i) need not be defined.
c
c     nbd is an INTEGER array of dimension n that must be set by the
c       user to the type of bounds imposed on the variables:
c       nbd(i)=0 if x(i) is unbounded,
c              1 if x(i) has only a lower bound,
c              2 if x(i) has both lower and upper bounds,
c              3 if x(i) has only an upper bound.
c
c     f is a DOUBLE PRECISION variable.  If the routine setulb returns
c       with task(1:2)= 'FG', then f must be set by the user to
c       contain the value of the function at the point x.
c
c     g is a DOUBLE PRECISION array of length n.  If the routine setulb
c       returns with taskb(1:2)= 'FG', then g must be set by the user to
c       contain the components of the gradient at the point x.
c
c     factr is a DOUBLE PRECISION variable that must be set by the user.
c       It is a tolerance in the termination test for the algorithm.
c       The iteration will stop when
c
c        (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr*epsmch
c
c       where epsmch is the machine precision which is automatically
c       generated by the code. Typical values for factr on a computer
c        with 15 digits of accuracy in double precision are:
c        factr=1.d+12 for low accuracy;
c              1.d+7  for moderate accuracy;
c              1.d+1  for extremely high accuracy.
c       The user can suppress this termination test by setting factr=0.
c
c     pgtol is a double precision variable.
c       On entry pgtol >= 0 is specified by the user.  The iteration
c         will stop when
c
c                 max{|proj g_i | i = 1, ..., n} <= pgtol
c
c         where pg_i is the ith component of the projected gradient.
c       The user can suppress this termination test by setting pgtol=0.
c
c     wa is a DOUBLE PRECISION  array of length
c       (2mmax + 5)nmax + 11mmax^2 + 8mmax used as workspace.
c       This array must not be altered by the user.
c
c     iwa is an INTEGER  array of length 3nmax used as
c       workspace. This array must not be altered by the user.
c
c     task is a CHARACTER string of length 60.
c       On first entry, it must be set to 'START'.
c       On a return with task(1:2)='FG', the user must evaluate the
c         function f and gradient g at the returned value of x.
c       On a return with task(1:5)='NEW_X', an iteration of the
c         algorithm has concluded, and f and g contain f(x) and g(x)
c         respectively.  The user can decide whether to continue or stop
c         the iteration.
c       When
c         task(1:4)='CONV', the termination test in L-BFGS-B has been
c           satisfied;
c         task(1:4)='ABNO', the routine has terminated abnormally
c           without being able to satisfy the termination conditions,
c           x contains the best approximation found,
c           f and g contain f(x) and g(x) respectively;
c         task(1:5)='ERROR', the routine has detected an error in the
c           input parameters;
c       On exit with task = 'CONV', 'ABNO' or 'ERROR', the variable task
c         contains additional information that the user can print.
c       This array should not be altered unless the user wants to
c          stop the run for some reason.  See driver2 or driver3
```

```
143 c           for a detailed explanation on how to stop the run
144 c           by assigning task(1:4)='STOP' in the driver.
145 c
146 c     iprint is an INTEGER variable that must be set by the user.
147 c       It controls the frequency and type of output generated:
148 c        iprint<0    no output is generated;
149 c        iprint=0    print only one line at the last iteration;
150 c        0<iprint<99 print also f and |proj g| every iprint iterations;
151 c        iprint=99   print details of every iteration except n-vectors;
152 c        iprint=100  print also the changes of active set and final x;
153 c        iprint>100  print details of every iteration including x and g;
154 c       When iprint > 0, the file iterate.dat will be created to
155 c                        summarize the iteration.
156 c
157 c     csave  is a CHARACTER working array of length 60.
158 c
159 c     lsave is a LOGICAL working array of dimension 4.
160 c       On exit with task = 'NEW_X', the following information is
161 c         available:
162 c       lsave(1) = .true.  the initial x did not satisfy the bounds;
163 c       lsave(2) = .true.  the problem contains bounds;
164 c       lsave(3) = .true.  each variable has upper and lower bounds.
165 c
166 c     isave is an INTEGER working array of dimension 44.
167 c       On exit with task = 'NEW_X', it contains information that
168 c       the user may want to access:
169 c         isave(30) = the current iteration number;
170 c         isave(34) = the total number of function and gradient
171 c                           evaluations;
172 c         isave(36) = the number of function value or gradient
173 c                                  evaluations in the current iteration;
174 c         isave(38) = the number of free variables in the current
175 c                           iteration;
176 c         isave(39) = the number of active constraints at the current
177 c                           iteration;
178 c
179 c         see the subroutine setulb.f for a description of other
180 c         information contained in isave
181 c
182 c     dsave is a DOUBLE PRECISION working array of dimension 29.
183 c       On exit with task = 'NEW_X', it contains information that
184 c         the user may want to access:
185 c         dsave(2) = the value of f at the previous iteration;
186 c         dsave(5) = the machine precision epsmch generated by the code;
187 c         dsave(13) = the infinity norm of the projected gradient;
188 c
189 c         see the subroutine setulb.f for a description of other
190 c         information contained in dsave
191 c
192 c     ------------------------------------------------------------
193 c           END OF THE DESCRIPTION OF THE VARIABLES IN L-BFGS-B
194 c     ------------------------------------------------------------
195
196       program driver
197       use json_writer, only: json_write_aggregate
198
199 c     This simple driver demonstrates how to call the L-BFGS-B code to
200 c       solve a sample problem (the extended Rosenbrock function
201 c       subject to bounds on the variables). The dimension n of this
202 c       problem is variable.
203
204       integer          nmax, mmax
205       parameter(nmax=1024, mmax=17)
206 c        nmax is the dimension of the largest problem to be solved.
207 c        mmax is the maximum number of limited memory corrections.
208
209 c     Declare the variables needed by the code.
210 c       A description of all these variables is given at the end of
211 c       the driver.
212
213       character*60     task, csave
214       logical          lsave(4)
215       integer          n, m, iprint,
216      +                 nbd(nmax), iwa(3*nmax), isave(44)
217       double precision f, factr, pgtol,
218      +                 x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
219      +                 wa(2*mmax*nmax + 5*nmax + 11*mmax*mmax + 8*mmax)
220
221 c     Declare a few additional variables for this sample problem.
222
223       double precision t1, t2
224       integer          i
225
226 c     Test instrumentation: dump per-iteration state to JSON when the
227 c     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
228       character*512    lbfgsb_json
229       logical          json_active
```

```
230
231 c     We wish to have output at every iteration.
232
233       iprint = 1
234
235 c     We specify the tolerances in the stopping criteria.
236
237       factr=1.0d+7
238       pgtol=1.0d-5
239
240 c     We specify the dimension n of the sample problem and the number
241 c        m of limited memory corrections stored.  (n and m should not
242 c        exceed the limits nmax and mmax respectively.)
243
244       n=25
245       m=5
246
247 c     We now provide nbd which defines the bounds on the variables:
248 c                    l   specifies the lower bounds,
249 c                    u   specifies the upper bounds.
250
251 c     First set bounds on the odd-numbered variables.
252
253       do 10 i=1,n,2
254          nbd(i)=2
255          l(i)=1.0d0
256          u(i)=1.0d2
257    10 continue
258
259 c     Next set bounds on the even-numbered variables.
260
261       do 12 i=2,n,2
262          nbd(i)=2
263          l(i)=-1.0d2
264          u(i)=1.0d2
265    12  continue
266
267 c     We now define the starting point.
268
269       do 14 i=1,n
270          x(i)=3.0d0
271    14 continue
272
273       write (6,16)
274    16 format(/,5x, 'Solving sample problem.',
275      +       /,5x, ' (f = 0.0 at the optimal solution.)',/)
276
277 c     We start the iteration by initializing task.
278 c
279       task = 'START'
280
281 c     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
282       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
283       json_active = (len_trim(lbfgsb_json) .gt. 0)
284
285 c        ------- the beginning of the loop ----------
286
287  111  continue
288
289 c     This is the call to the L-BFGS-B code.
290
291       call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
292      +            csave,lsave,isave,dsave)
293
294       if (task(1:2) .eq. 'FG') then
295 c        the minimization routine has returned to request the
296 c        function f and gradient g values at the current x.
297
298 c        Compute function value f for the sample problem.
299
300          f=.25d0*(x(1)-1.d0)**2
301          do 20 i=2,n
302             f=f+(x(i)-x(i-1)**2)**2
303    20    continue
304          f=4.d0*f
305
306 c        Compute gradient g for the sample problem.
307
308          t1=x(2)-x(1)**2
309          g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
310          do 22 i=2,n-1
311             t2=t1
312             t1=x(i+1)-x(i)**2
313             g(i)=8.d0*t2-1.6d1*x(i)*t1
314    22    continue
315          g(n)=8.d0*t1
316
```

```
317 c          go back to the minimization routine.
318          goto 111
319       endif
320 c
321       if (task(1:5) .eq. 'NEW_X')  goto 111
322 c        the minimization routine has returned with a new iterate,
323 c         and we have opted to continue the iteration.
324
325 c           ---------- the end of the loop -------------
326
327 c     If task is neither FG nor NEW_X we terminate execution.
328
329       if (json_active) then
330          call json_write_aggregate(trim(lbfgsb_json), task, f,
331      +                             dsave(13), n, x)
332       endif
333
334       stop
335
336       end
337
338 c======================= The end of driver1 ============================
```

13.2 driver1.f90

This simple driver demonstrates how to call the L-BFGS-B code to solve a sample problem (the extended Rosenbrock function subject to bounds on the variables). The dimension n of this problem is variable. (Fortran-90 version)

```
1 !> \file driver1.f90
2
3 !
4 !  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 !  or "3-clause license")
6 !  Please read attached file License.txt
7 !
8 !
9 !                     DRIVER1 in Fortran 90
10 !     ----------------------------------------------------------------
11 !
12 !        L-BFGS-B is a code for solving large nonlinear optimization
13 !             problems with simple bounds on the variables.
14 !
15 !        The code can also be used for unconstrained problems and is
16 !        as efficient for these problems as the earlier limited memory
17 !                         code L-BFGS.
18 !
19 !        This is the simplest driver in the package. It uses all the
20 !                    default settings of the code.
21 !
22 !
23 !     References:
24 !
25 !        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
26 !        memory algorithm for bound constrained optimization",
27 !        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
28 !
29 !        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
30 !        Subroutines for Large Scale Bound Constrained Optimization"
31 !        Tech. Report, NAM-11, EECS Department, Northwestern University,
32 !        1994.
33 !
34 !
35 !          (Postscript files of these papers are available via anonymous
36 !           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
37 !
38 !                               *  *  *
39 !
40 !         March 2011   (latest revision)
41 !         Optimization Center at Northwestern University
42 !         Instituto Tecnologico Autonomo de Mexico
43 !
44 !         Jorge Nocedal and Jose Luis Morales
45 !
46 !     ----------------------------------------------------------------
47 !            DESCRIPTION OF THE VARIABLES IN L-BFGS-B
48 !     ----------------------------------------------------------------
49 !
50 !     n is an INTEGER variable that must be set by the user to the
51 !       number of variables.  It is not altered by the routine.
52 !
53 !     m is an INTEGER variable that must be set by the user to the
54 !       number of corrections used in the limited memory matrix.
55 !       It is not altered by the routine.  Values of m < 3  are
```

```
56 !       not recommended, and large values of m can result in excessive
57 !       computing time. The range  3 <= m <= 20 is recommended.
58 !
59 !     x is a DOUBLE PRECISION array of length n.  On initial entry
60 !       it must be set by the user to the values of the initial
61 !       estimate of the solution vector.  Upon successful exit, it
62 !       contains the values of the variables at the best point
63 !       found (usually an approximate solution).
64 !
65 !     l is a DOUBLE PRECISION array of length n that must be set by
66 !       the user to the values of the lower bounds on the variables. If
67 !       the i-th variable has no lower bound, l(i) need not be defined.
68 !
69 !     u is a DOUBLE PRECISION array of length n that must be set by
70 !       the user to the values of the upper bounds on the variables. If
71 !       the i-th variable has no upper bound, u(i) need not be defined.
72 !
73 !     nbd is an INTEGER array of dimension n that must be set by the
74 !       user to the type of bounds imposed on the variables:
75 !       nbd(i)=0 if x(i) is unbounded,
76 !              1 if x(i) has only a lower bound,
77 !              2 if x(i) has both lower and upper bounds,
78 !              3 if x(i) has only an upper bound.
79 !
80 !     f is a DOUBLE PRECISION variable.  If the routine setulb returns
81 !       with task(1:2)= 'FG', then f must be set by the user to
82 !       contain the value of the function at the point x.
83 !
84 !     g is a DOUBLE PRECISION array of length n.  If the routine setulb
85 !       returns with taskb(1:2)= 'FG', then g must be set by the user to
86 !       contain the components of the gradient at the point x.
87 !
88 !     factr is a DOUBLE PRECISION variable that must be set by the user.
89 !       It is a tolerance in the termination test for the algorithm.
90 !       The iteration will stop when
91 !
92 !        (f^k - f^{k+1})/max{|f^k|,|f^{k+1}|,1} <= factr*epsmch
93 !
94 !       where epsmch is the machine precision which is automatically
95 !       generated by the code. Typical values for factr on a computer
96 !       with 15 digits of accuracy in double precision are:
97 !       factr=1.d+12 for low accuracy;
98 !             1.d+7  for moderate accuracy;
99 !             1.d+1  for extremely high accuracy.
100 !       The user can suppress this termination test by setting factr=0.
101 !
102 !     pgtol is a double precision variable.
103 !       On entry pgtol >= 0 is specified by the user.  The iteration
104 !         will stop when
105 !
106 !                 max{|proj g_i | i = 1, ..., n} <= pgtol
107 !
108 !         where pg_i is the ith component of the projected gradient.
109 !       The user can suppress this termination test by setting pgtol=0.
110 !
111 !     wa is a DOUBLE PRECISION  array of length
112 !       (2mmax + 5)nmax + 11mmax^2 + 8mmax used as workspace.
113 !       This array must not be altered by the user.
114 !
115 !     iwa is an INTEGER  array of length 3nmax used as
116 !       workspace. This array must not be altered by the user.
117 !
118 !     task is a CHARACTER string of length 60.
119 !       On first entry, it must be set to 'START'.
120 !       On a return with task(1:2)='FG', the user must evaluate the
121 !         function f and gradient g at the returned value of x.
122 !       On a return with task(1:5)='NEW_X', an iteration of the
123 !         algorithm has concluded, and f and g contain f(x) and g(x)
124 !         respectively.  The user can decide whether to continue or stop
125 !         the iteration.
126 !       When
127 !         task(1:4)='CONV', the termination test in L-BFGS-B has been
128 !           satisfied;
129 !         task(1:4)='ABNO', the routine has terminated abnormally
130 !           without being able to satisfy the termination conditions,
131 !           x contains the best approximation found,
132 !           f and g contain f(x) and g(x) respectively;
133 !         task(1:5)='ERROR', the routine has detected an error in the
134 !           input parameters;
135 !       On exit with task = 'CONV', 'ABNO' or 'ERROR', the variable task
136 !         contains additional information that the user can print.
137 !       This array should not be altered unless the user wants to
138 !          stop the run for some reason.  See driver2 or driver3
139 !          for a detailed explanation on how to stop the run
140 !          by assigning task(1:4)='STOP' in the driver.
141 !
142 !     iprint is an INTEGER variable that must be set by the user.
```

```
!       It controls the frequency and type of output generated:
!        iprint<0    no output is generated;
!        iprint=0    print only one line at the last iteration;
!        0<iprint<99 print also f and |proj g| every iprint iterations;
!        iprint=99   print details of every iteration except n-vectors;
!        iprint=100  print also the changes of active set and final x;
!        iprint>100  print details of every iteration including x and g;
!       When iprint > 0, the file iterate.dat will be created to
!                        summarize the iteration.
!
!     csave  is a CHARACTER working array of length 60.
!
!     lsave is a LOGICAL working array of dimension 4.
!       On exit with task = 'NEW_X', the following information is
!         available:
!       lsave(1) = .true.  the initial x did not satisfy the bounds;
!       lsave(2) = .true.  the problem contains bounds;
!       lsave(3) = .true.  each variable has upper and lower bounds.
!
!     isave is an INTEGER working array of dimension 44.
!       On exit with task = 'NEW_X', it contains information that
!       the user may want to access:
!         isave(30) = the current iteration number;
!         isave(34) = the total number of function and gradient
!                           evaluations;
!         isave(36) = the number of function value or gradient
!                                  evaluations in the current iteration;
!         isave(38) = the number of free variables in the current
!                           iteration;
!         isave(39) = the number of active constraints at the current
!                           iteration;
!
!         see the subroutine setulb.f for a description of other
!         information contained in isave
!
!     dsave is a DOUBLE PRECISION working array of dimension 29.
!       On exit with task = 'NEW_X', it contains information that
!         the user may want to access:
!         dsave(2)  = the value of f at the previous iteration;
!         dsave(5)  = the machine precision epsmch generated by the code;
!         dsave(13) = the infinity norm of the projected gradient;
!
!         see the subroutine setulb.f for a description of other
!         information contained in dsave
!
!     ------------------------------------------------------------
!           END OF THE DESCRIPTION OF THE VARIABLES IN L-BFGS-B
!     ------------------------------------------------------------
!
      program driver
      use json_writer, only: json_write_aggregate
!
!     This simple driver demonstrates how to call the L-BFGS-B code to
!       solve a sample problem (the extended Rosenbrock function
!       subject to bounds on the variables). The dimension n of this
!       problem is variable.

      implicit none
!
!     Declare variables and parameters needed by the code.
!       Note thar we wish to have output at every iteration.
!          iprint=1
!
!       We also specify the tolerances in the stopping criteria.
!          factr  = 1.0d+7, pgtol  = 1.0d-5
!
!       A description of all these variables is given at the beginning
!       of  the driver
!
      integer,  parameter    :: n = 25, m = 5, iprint = 1
      integer,  parameter    :: dp = kind(1.0d0)
      real(dp), parameter    :: factr  = 1.0d+7, pgtol  = 1.0d-5
!
      character(len=60)      :: task, csave
      logical                :: lsave(4)
      integer                :: isave(44)
      real(dp)               :: f
      real(dp)               :: dsave(29)
      integer,  allocatable  :: nbd(:), iwa(:)
      real(dp), allocatable  :: x(:), l(:), u(:), g(:), wa(:)

!     Declare a few additional variables for this sample problem

      real(dp)               :: t1, t2
      integer                :: i

!     Test instrumentation: dump per-iteration state to JSON when the
```

```
230 !     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
231       character(len=512)     :: lbfgsb_json
232       logical                :: json_active
233
234 !     Allocate dynamic arrays
235
236       allocate ( nbd(n), x(n), l(n), u(n), g(n) )
237       allocate ( iwa(3*n) )
238       allocate ( wa(2*m*n + 11*m*m + 5*n + 8*m) )
239 !
240       do 10 i=1, n, 2
241          nbd(i) = 2
242          l(i)   = 1.0d0
243          u(i)   = 1.0d2
244    10 continue
245
246 !     Next set bounds on the even-numbered variables.
247
248       do 12 i=2, n, 2
249          nbd(i) =  2
250          l(i)   = -1.0d2
251          u(i)   =  1.0d2
252    12  continue
253
254 !     We now define the starting point.
255
256       do 14 i=1, n
257          x(i) = 3.0d0
258    14 continue
259
260       write (6,16)
261    16 format(/,5x, 'Solving sample problem.', &
262              /,5x, ' (f = 0.0 at the optimal solution.)',/)
263
264 !     We start the iteration by initializing task.
265
266       task = 'START'
267
268 !     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
269       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
270       json_active = (len_trim(lbfgsb_json) > 0)
271
272 !     The beginning of the loop
273
274       do while(task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &
275                task.eq.'START')
276
277 !     This is the call to the L-BFGS-B code.
278
279          call setulb ( n, m, x, l, u, nbd, f, g, factr, pgtol, &
280                        wa, iwa, task, iprint,&
281                        csave, lsave, isave, dsave )
282
283          if (task(1:2) .eq. 'FG') then
284
285             f=.25d0*( x(1)-1.d0 )**2
286             do 20 i=2, n
287                f = f + ( x(i)-x(i-1 )**2 )**2
288    20       continue
289             f = 4.d0*f
290
291 !     Compute gradient g for the sample problem.
292
293             t1   = x(2) - x(1)**2
294             g(1) = 2.d0*(x(1) - 1.d0) - 1.6d1*x(1)*t1
295             do 22 i=2, n-1
296                t2   = t1
297                t1   = x(i+1) - x(i)**2
298                g(i) = 8.d0*t2 - 1.6d1*x(i)*t1
299    22       continue
300             g(n) = 8.d0*t1
301
302          end if
303       end do
304
305 !     end of loop do while
306
307       if (json_active) &
308          call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13), n, x)
309
310       end program driver
311
```

13.3 driver2.f

This driver shows how to replace the default stopping test by other termination criteria. It also illustrates how to print the values of several parameters during the course of the iteration. The sample problem used here is the same as in DRIVER1 (the extended Rosenbrock function with bounds on the variables). (Fortran-77 version)

```
1 c> \file driver2.f
2
3 c
4 c  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c  or "3-clause license")
6 c  Please read attached file License.txt
7 c
8 c                           DRIVER 2 in Fortran 77
9 c     ----------------------------------------------------------------
10 c             CUSTOMIZED DRIVER FOR L-BFGS-B (version 3.0)
11 c     ----------------------------------------------------------------
12 c
13 c        L-BFGS-B is a code for solving large nonlinear optimization
14 c             problems with simple bounds on the variables.
15 c
16 c        The code can also be used for unconstrained problems and is
17 c        as efficient for these problems as the earlier limited memory
18 c                         code L-BFGS.
19 c
20 c        This driver illustrates how to control the termination of the
21 c        run and how to design customized output.
22 c
23 c     References:
24 c
25 c        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
26 c        memory algorithm for bound constrained optimization",
27 c        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
28 c
29 c        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
30 c        Subroutines for Large Scale Bound Constrained Optimization"
31 c        Tech. Report, NAM-11, EECS Department, Northwestern University,
32 c        1994.
33 c
34 c
35 c          (Postscript files of these papers are available via anonymous
36 c           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
37 c
38 c                               *  *  *
39 c
40 c         February 2011   (latest revision)
41 c         Optimization Center at Northwestern University
42 c         Instituto Tecnologico Autonomo de Mexico
43 c
44 c         Jorge Nocedal and Jose Luis Morales
45 c         Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
46 c         L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
47 c         Optimization"  (2011). To appear in  ACM Transactions on
48 c         Mathematical Software,
49 c
50 c     **************
51
52       program driver
53       use json_writer, only: json_write_aggregate
54
55 c     This driver shows how to replace the default stopping test
56 c       by other termination criteria. It also illustrates how to
57 c       print the values of several parameters during the course of
58 c       the iteration. The sample problem used here is the same as in
59 c       DRIVER1 (the extended Rosenbrock function with bounds on the
60 c       variables).
61
62       integer          nmax, mmax
63       parameter(nmax=1024, mmax=17)
64 c        nmax is the dimension of the largest problem to be solved.
65 c        mmax is the maximum number of limited memory corrections.
66
67 c     Declare the variables needed by the code.
68 c       A description of all these variables is given at the end of
69 c       driver1.
70
71       character*60     task, csave
72       logical          lsave(4)
73       integer          n, m, iprint,
74      +                 nbd(nmax), iwa(3*nmax), isave(44)
75       double precision f, factr, pgtol,
76      +                 x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
77      +                 wa(2*mmax*nmax+5*nmax+11*mmax*mmax+8*mmax)
78
79 c     Declare a few additional variables for the sample problem.
80
```

```
81       double precision t1, t2
82       integer          i
83
84 c     Test instrumentation: dump aggregate end-of-run state to JSON when
85 c     the environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
86 c     LBFGSB_NFG_LIMIT optionally overrides the nfg stopping threshold.
87       character*512    lbfgsb_json
88       character*64     env_nfg_limit
89       logical          json_active
90       integer          nfg_limit
91
92 c     We suppress the default output.
93
94       iprint = -1
95
96 c     We suppress both code-supplied stopping tests because the
97 c        user is providing his own stopping criteria.
98
99       factr=0.0d0
100       pgtol=0.0d0
101
102 c     We specify the dimension n of the sample problem and the number
103 c        m of limited memory corrections stored.  (n and m should not
104 c        exceed the limits nmax and mmax respectively.)
105
106       n=25
107       m=5
108
109 c     We now specify nbd which defines the bounds on the variables:
110 c                     l   specifies the lower bounds,
111 c                     u   specifies the upper bounds.
112
113 c     First set bounds on the odd numbered variables.
114
115       do 10 i=1,n,2
116          nbd(i)=2
117          l(i)=1.0d0
118          u(i)=1.0d2
119    10 continue
120
121 c     Next set bounds on the even numbered variables.
122
123       do 12 i=2,n,2
124          nbd(i)=2
125          l(i)=-1.0d2
126          u(i)=1.0d2
127    12  continue
128
129 c     We now define the starting point.
130
131       do 14 i=1,n
132          x(i)=3.0d0
133    14 continue
134
135 c     We now write the heading of the output.
136
137       write (6,16)
138    16 format(/,5x, 'Solving sample problem.',
139      +       /,5x, ' (f = 0.0 at the optimal solution.)',/)
140
141 c     We start the iteration by initializing task.
142 c
143       task = 'START'
144
145 c     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
146       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
147       json_active = (len_trim(lbfgsb_json) .gt. 0)
148       nfg_limit = 99
149       call get_environment_variable('LBFGSB_NFG_LIMIT', env_nfg_limit)
150       if (len_trim(env_nfg_limit) .gt. 0) then
151          read(env_nfg_limit, *) nfg_limit
152       endif
153
154 c        ------- the beginning of the loop ----------
155
156  111  continue
157
158 c     This is the call to the L-BFGS-B code.
159
160       call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
161      +            csave,lsave,isave,dsave)
162
163       if (task(1:2) .eq. 'FG') then
164 c        the minimization routine has returned to request the
165 c        function f and gradient g values at the current x.
166
167 c        Compute function value f for the sample problem.
```

```
168
169           f=.25d0*(x(1)-1.d0)**2
170           do 20 i=2,n
171              f=f+(x(i)-x(i-1)**2)**2
172     20    continue
173           f=4.d0*f
174
175   c       Compute gradient g for the sample problem.
176
177           t1=x(2)-x(1)**2
178           g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
179           do 22 i=2,n-1
180              t2=t1
181              t1=x(i+1)-x(i)**2
182              g(i)=8.d0*t2-1.6d1*x(i)*t1
183     22    continue
184           g(n)=8.d0*t1
185
186   c         go back to the minimization routine.
187           goto 111
188         endif
189   c
190         if (task(1:5) .eq. 'NEW_X') then
191   c
192   c        the minimization routine has returned with a new iterate.
193   c        At this point have the opportunity of stopping the iteration
194   c        or observing the values of certain parameters
195   c
196   c        First are two examples of stopping tests.
197
198   c        Note: task(1:4) must be assigned the value 'STOP' to terminate
199   c          the iteration and ensure that the final results are
200   c          printed in the default format. The rest of the character
201   c          string TASK may be used to store other information.
202
203   c        1) Terminate if the total number of f and g evaluations
204   c             exceeds 99.
205
206           if (isave(34) .ge. nfg_limit)
207        +     task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
208
209   c        2) Terminate if  |proj g|/(1+|f|) < 1.0d-10, where
210   c           "proj g" denoted the projected gradient
211
212           if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f)))
213        +     task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
214
215   c        We now wish to print the following information at each
216   c        iteration:
217   c
218   c          1) the current iteration number, isave(30),
219   c          2) the total number of f and g evaluations, isave(34),
220   c          3) the value of the objective function f,
221   c          4) the norm of the projected gradient,  dsve(13)
222   c
223   c        See the comments at the end of driver1 for a description
224   c        of the variables isave and dsave.
225
226           write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate'
227        +     ,isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
228
229   c        If the run is to be terminated, we print also the information
230   c        contained in task as well as the final value of x.
231
232           if (task(1:4) .eq. 'STOP') then
233              write (6,*) task
234              write (6,*) 'Final X='
235              write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
236           endif
237
238   c         go back to the minimization routine.
239           goto 111
240
241         endif
242
243   c           ---------- the end of the loop -------------
244
245   c     If task is neither FG nor NEW_X we terminate execution.
246
247         if (json_active) then
248            call json_write_aggregate(trim(lbfgsb_json), task, f,
249        +                             dsave(13), n, x)
250         endif
251
252         stop
253
254         end
```

```
255
256 c======================= The end of driver2 ============================
257
```

13.4 driver2.f90

This driver shows how to replace the default stopping test by other termination criteria. It also illustrates how to print the values of several parameters during the course of the iteration. The sample problem used here is the same as in DRIVER1 (the extended Rosenbrock function with bounds on the variables). (Fortran-90 version)

```
1 !> \file driver2.f90
2
3 !
4 !  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 !  or "3-clause license")
6 !  Please read attached file License.txt
7 !
8 !
9 !                 DRIVER 2  in Fortran 90
10 !    ----------------------------------------------------------------
11 !             CUSTOMIZED DRIVER FOR L-BFGS-B
12 !    ----------------------------------------------------------------
13 !
14 !       L-BFGS-B is a code for solving large nonlinear optimization
15 !            problems with simple bounds on the variables.
16 !
17 !       The code can also be used for unconstrained problems and is
18 !       as efficient for these problems as the earlier limited memory
19 !                         code L-BFGS.
20 !
21 !       This driver illustrates how to control the termination of the
22 !       run and how to design customized output.
23 !
24 !    References:
25 !
26 !       [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 !       memory algorithm for bound constrained optimization",
28 !       SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 !
30 !       [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 !       Subroutines for Large Scale Bound Constrained Optimization"
32 !       Tech. Report, NAM-11, EECS Department, Northwestern University,
33 !       1994.
34 !
35 !
36 !         (Postscript files of these papers are available via anonymous
37 !          ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 !
39 !                              *  *  *
40 !
41 !         February 2011   (latest revision)
42 !         Optimization Center at Northwestern University
43 !         Instituto Tecnologico Autonomo de Mexico
44 !
45 !         Jorge Nocedal and Jose Luis Morales
46 !
47 !    **************
48       program driver
49       use json_writer, only: json_write_aggregate
50
51 !     This driver shows how to replace the default stopping test
52 !     by other termination criteria. It also illustrates how to
53 !     print the values of several parameters during the course of
54 !     the iteration. The sample problem used here is the same as in
55 !     DRIVER1 (the extended Rosenbrock function with bounds on the
56 !     variables).
57
58       implicit none
59
60 !     Declare variables and parameters needed by the code.
61 !
62 !     Note that we suppress the default output (iprint = -1)
63 !     We suppress both code-supplied stopping tests because the
64 !     user is providing his/her own stopping criteria.
65
66       integer,  parameter    :: n = 25, m = 5, iprint = -1
67       integer,  parameter    :: dp = kind(1.0d0)
68       real(dp), parameter    :: factr  = 0.0d0, pgtol  = 0.0d0
69
70       character(len=60)      :: task, csave
71       logical                :: lsave(4)
72       integer                :: isave(44)
```

```
73       real(dp)               :: f
74       real(dp)               :: dsave(29)
75       integer,  allocatable  :: nbd(:), iwa(:)
76       real(dp), allocatable  :: x(:), l(:), u(:), g(:), wa(:)
77 !
78       real(dp)               :: t1, t2
79       integer                :: i
80
81 !     Test instrumentation: dump aggregate end-of-run state to JSON when
82 !     the environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
83 !     LBFGSB_NFG_LIMIT optionally overrides the nfg stopping threshold.
84       character(len=512)     :: lbfgsb_json
85       character(len=64)      :: env_nfg_limit
86       logical                :: json_active
87       integer                :: nfg_limit
88
89       allocate ( nbd(n), x(n), l(n), u(n), g(n) )
90       allocate ( iwa(3*n) )
91       allocate ( wa(2*m*n + 5*n + 11*m*m + 8*m) )
92 !
93 !    This driver shows how to replace the default stopping test
94 !    by other termination criteria. It also illustrates how to
95 !    print the values of several parameters during the course of
96 !    the iteration. The sample problem used here is the same as in
97 !    DRIVER1 (the extended Rosenbrock function with bounds on the
98 !    variables).
99 !    We now specify nbd which defines the bounds on the variables:
100 !                   l   specifies the lower bounds,
101 !                   u   specifies the upper bounds.
102
103 !    First set bounds on the odd numbered variables.
104
105       do 10 i=1, n,2
106          nbd(i)=2
107          l(i)=1.0d0
108          u(i)=1.0d2
109   10  continue
110
111 !    Next set bounds on the even numbered variables.
112
113       do 12 i=2, n,2
114          nbd(i)=2
115          l(i)=-1.0d2
116          u(i)=1.0d2
117   12   continue
118
119 !     We now define the starting point.
120
121       do 14 i=1, n
122          x(i)=3.0d0
123   14  continue
124
125 !     We now write the heading of the output.
126
127       write (6,16)
128   16  format(/,5x, 'Solving sample problem.', &
129              /,5x, ' (f = 0.0 at the optimal solution.)',/)
130
131
132 !     We start the iteration by initializing task.
133 !
134       task = 'START'
135
136 !     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
137       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
138       json_active = (len_trim(lbfgsb_json) > 0)
139       nfg_limit = 99
140       call get_environment_variable('LBFGSB_NFG_LIMIT', env_nfg_limit)
141       if (len_trim(env_nfg_limit) > 0) read(env_nfg_limit, *) nfg_limit
142
143 !        ------- the beginning of the loop ----------
144
145       do while( task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &
146                 task.eq.'START')
147
148 !     This is the call to the L-BFGS-B code.
149
150          call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint, &
151                      csave,lsave,isave,dsave)
152
153          if (task(1:2) .eq. 'FG') then
154
155 !        the minimization routine has returned to request the
156 !        function f and gradient g values at the current x.
157
158 !        Compute function value f for the sample problem.
159
```

```
160             f =.25d0*(x(1) - 1.d0)**2
161             do 20 i=2,n
162                f = f + (x(i) - x(i-1)**2)**2
163   20        continue
164             f = 4.d0*f
165
166 !        Compute gradient g for the sample problem.
167
168             t1 = x(2) - x(1)**2
169             g(1) = 2.d0*(x(1) - 1.d0) - 1.6d1*x(1)*t1
170             do 22 i= 2,n-1
171                t2 = t1
172                t1 = x(i+1) - x(i)**2
173                g(i) = 8.d0*t2 - 1.6d1*x(i)*t1
174   22        continue
175             g(n)=8.d0*t1
176 !
177       else
178 !
179          if (task(1:5) .eq. 'NEW_X') then
180 !
181 !       the minimization routine has returned with a new iterate.
182 !       At this point have the opportunity of stopping the iteration
183 !       or observing the values of certain parameters
184 !
185 !       First are two examples of stopping tests.
186
187 !       Note: task(1:4) must be assigned the value 'STOP' to terminate
188 !         the iteration and ensure that the final results are
189 !         printed in the default format. The rest of the character
190 !         string TASK may be used to store other information.
191
192 !       1) Terminate if the total number of f and g evaluations
193 !            exceeds 99.
194
195              if (isave(34) .ge. nfg_limit) &
196                 task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
197
198 !       2) Terminate if  |proj g|/(1+|f|) < 1.0d-10, where
199 !          "proj g" denoted the projected gradient
200
201              if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f))) &
202                task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
203
204 !       We now wish to print the following information at each
205 !       iteration:
206 !
207 !         1) the current iteration number, isave(30),
208 !         2) the total number of f and g evaluations, isave(34),
209 !         3) the value of the objective function f,
210 !         4) the norm of the projected gradient,  dsve(13)
211 !
212 !       See the comments at the end of driver1 for a description
213 !       of the variables isave and dsave.
214
215             write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate' &
216                , isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
217
218 !        If the run is to be terminated, we print also the information
219 !        contained in task as well as the final value of x.
220
221             if (task(1:4) .eq. 'STOP') then
222                write (6,*) task
223                write (6,*) 'Final X='
224                write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
225             end if
226
227          end if
228       end if
229
230       end do
231 !           ---------- the end of the loop -------------
232
233 !     If task is neither FG nor NEW_X we terminate execution.
234
235       if (json_active) &
236          call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13), n, x)
237
238       end program driver
239
240 !======================= The end of driver2 ============================
241
```

13.5 driver3.f

This time-controlled driver shows that it is possible to terminate a run by elapsed CPU time, and yet be able to print all desired information. This driver also illustrates the use of two stopping criteria that may be used in conjunction with a limit on execution time. The sample problem used here is the same as in driver1 and driver2 (the extended Rosenbrock function with bounds on the variables). (Fortran-77 version)

```
1 c> \file driver3.f
2
3 c
4 c  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 c  or "3-clause license")
6 c  Please read attached file License.txt
7 c
8 c                           DRIVER 3 in Fortran 77
9 c     ------------------------------------------------------------
10 c           TIME-CONTROLLED DRIVER FOR L-BFGS-B (version 3.0)
11 c     ------------------------------------------------------------
12 c
13 c        L-BFGS-B is a code for solving large nonlinear optimization
14 c             problems with simple bounds on the variables.
15 c
16 c        The code can also be used for unconstrained problems and is
17 c        as efficient for these problems as the earlier limited memory
18 c                         code L-BFGS.
19 c
20 c        This driver shows how to terminate a run after some prescribed
21 c        CPU time has elapsed, and how to print the desired information
22 c        before exiting.
23 c
24 c     References:
25 c
26 c        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 c        memory algorithm for bound constrained optimization",
28 c        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 c
30 c        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 c        Subroutines for Large Scale Bound Constrained Optimization"
32 c        Tech. Report, NAM-11, EECS Department, Northwestern University,
33 c        1994.
34 c
35 c
36 c          (Postscript files of these papers are available via anonymous
37 c           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
38 c
39 c                               *  *  *
40 c
41 c         February 2011   (latest revision)
42 c         Optimization Center at Northwestern University
43 c         Instituto Tecnologico Autonomo de Mexico
44 c
45 c         Jorge Nocedal and Jose Luis Morales, Remark on "Algorithm 778:
46 c         L-BFGS-B: Fortran Subroutines for Large-Scale Bound Constrained
47 c         Optimization"  (2011). To appear in  ACM Transactions on
48 c         Mathematical Software,
49 c
50 c
51 c     **************
52
53       program driver
54       use json_writer, only: json_write_aggregate
55
56 c     This time-controlled driver shows that it is possible to terminate
57 c     a run by elapsed CPU time, and yet be able to print all desired
58 c     information. This driver also illustrates the use of two
59 c     stopping criteria that may be used in conjunction with a limit
60 c     on execution time. The sample problem used here is the same as in
61 c     driver1 and driver2 (the extended Rosenbrock function with bounds
62 c     on the variables).
63
64       integer          nmax, mmax
65       parameter(nmax=1024,mmax=17)
66 c        nmax is the dimension of the largest problem to be solved.
67 c        mmax is the maximum number of limited memory corrections.
68
69 c     Declare the variables needed by the code.
70 c       A description of all these variables is given at the end of
71 c       driver1.
72
73       character*60     task, csave
74       logical          lsave(4)
75       integer          n, m, iprint,
76      +                 nbd(nmax), iwa(3*nmax), isave(44)
77       double precision f, factr, pgtol,
78      +                 x(nmax), l(nmax), u(nmax), g(nmax), dsave(29),
```

```
79      +                 wa(2*mmax*nmax+5*nmax+11*mmax*mmax+8*mmax)
80
81 c     Declare a few additional variables for the sample problem
82 c       and for keeping track of time.
83
84       double precision t1, t2, time1, time2, tlimit
85       integer          i, j
86
87 c     Test instrumentation: dump per-iteration state to JSON when the
88 c     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
89 c     LBFGSB_TLIMIT optionally overrides tlimit for reproducibility.
90       character*512    lbfgsb_json
91       character*64     env_tlimit
92       logical          json_active
93
94 c     We specify a limite on the CPU time (in seconds).
95
96       tlimit = 0.2
97       call get_environment_variable('LBFGSB_TLIMIT', env_tlimit)
98       if (len_trim(env_tlimit) .gt. 0) read(env_tlimit, *) tlimit
99
100 c     We suppress the default output.  (The user could also elect to
101 c       use the default output by choosing iprint >= 0.)
102
103       iprint = -1
104
105 c     We suppress the code-supplied stopping tests because we will
106 c       provide our own termination conditions
107
108       factr=0.0d0
109       pgtol=0.0d0
110
111 c     We specify the dimension n of the sample problem and the number
112 c        m of limited memory corrections stored.  (n and m should not
113 c        exceed the limits nmax and mmax respectively.)
114
115       n=1000
116       m=10
117
118 c     We now specify nbd which defines the bounds on the variables:
119 c                     l   specifies the lower bounds,
120 c                     u   specifies the upper bounds.
121
122 c     First set bounds on the odd-numbered variables.
123
124       do 10 i=1,n,2
125          nbd(i)=2
126          l(i)=1.0d0
127          u(i)=1.0d2
128    10 continue
129
130 c     Next set bounds on the even-numbered variables.
131
132       do 12 i=2,n,2
133          nbd(i)=2
134          l(i)=-1.0d2
135          u(i)=1.0d2
136    12  continue
137
138 c     We now define the starting point.
139
140       do 14 i=1,n
141          x(i)=3.0d0
142    14 continue
143
144 c     We now write the heading of the output.
145
146       write (6,16)
147    16 format(/,5x, 'Solving sample problem.',
148      +       /,5x, ' (f = 0.0 at the optimal solution.)',/)
149
150 c     We start the iteration by initializing task.
151 c
152       task = 'START'
153
154 c     Test instrumentation: enabled when LBFGSB_JSON_OUTPUT is set.
155       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
156       json_active = (len_trim(lbfgsb_json) .gt. 0)
157
158 c        ------- the beginning of the loop ----------
159
160 c     We begin counting the CPU time.
161
162       call timer(time1)
163
164  111  continue
165
```

```
166 c     This is the call to the L-BFGS-B code.
167
168       call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa,task,iprint,
169      +            csave,lsave,isave,dsave)
170
171       if (task(1:2) .eq. 'FG') then
172 c        the minimization routine has returned to request the
173 c        function f and gradient g values at the current x.
174 c        Before evaluating f and g we check the CPU time spent.
175
176          call timer(time2)
177          if (time2-time1 .gt. tlimit) then
178             task='STOP: CPU EXCEEDING THE TIME LIMIT.'
179
180 c          Note: Assigning task(1:4)='STOP' will terminate the run;
181 c          setting task(7:9)='CPU' will restore the information at
182 c          the latest iterate generated by the code so that it can
183 c          be correctly printed by the driver.
184
185 c          In this driver we have chosen to disable the
186 c          printing options of the code (we set iprint=-1);
187 c          instead we are using customized output: we print the
188 c          latest value of x, the corresponding function value f and
189 c          the norm of the projected gradient |proj g|.
190
191 c          We print out the information contained in task.
192
193             write (6,*) task
194
195 c          We print the latest iterate contained in wa(j+1:j+n), where
196 c
197             j = 3*n+2*m*n+11*m**2
198             write (6,*) 'Latest iterate X ='
199             write (6,'((1x,1p, 6(1x,d11.4)))') (wa(i),i = j+1,j+n)
200
201 c          We print the function value f and the norm of the projected
202 c          gradient |proj g| at the last iterate; they are stored in
203 c          dsave(2) and dsave(13) respectively.
204
205             write (6,'(a,1p,d12.5,4x,a,1p,d12.5)')
206      +         'At latest iterate   f =',dsave(2),'|proj g| =',dsave(13)
207
208          else
209
210 c          The time limit has not been reached and we compute
211 c          the function value f for the sample problem.
212
213             f=.25d0*(x(1)-1.d0)**2
214             do 20 i=2,n
215                f=f+(x(i)-x(i-1)**2)**2
216    20       continue
217             f=4.d0*f
218
219 c          Compute gradient g for the sample problem.
220
221             t1=x(2)-x(1)**2
222             g(1)=2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
223             do 22 i=2,n-1
224                t2=t1
225                t1=x(i+1)-x(i)**2
226                g(i)=8.d0*t2-1.6d1*x(i)*t1
227    22       continue
228             g(n)=8.d0*t1
229
230          endif
231
232 c          go back to the minimization routine.
233          goto 111
234       endif
235 c
236       if (task(1:5) .eq. 'NEW_X') then
237 c        the minimization routine has returned with a new iterate.
238 c        The time limit has not been reached, and we test whether
239 c        the following two stopping tests are satisfied:
240
241 c        1) Terminate if the total number of f and g evaluations
242 c             exceeds 900.
243
244          if (isave(34) .ge. 900)
245      +      task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
246
247 c        2) Terminate if  |proj g|/(1+|f|) < 1.0d-10.
248
249          if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f)))
250      +      task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
251
252 c        We wish to print the following information at each iteration:
```

```
253 c          1) the current iteration number, isave(30),
254 c          2) the total number of f and g evaluations, isave(34),
255 c          3) the value of the objective function f,
256 c          4) the norm of the projected gradient,  dsve(13)
257 c
258 c        See the comments at the end of driver1 for a description
259 c        of the variables isave and dsave.
260
261
262          write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate'
263      +      ,isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
264
265 c        If the run is to be terminated, we print also the information
266 c        contained in task as well as the final value of x.
267
268
269          if (task(1:4) .eq. 'STOP') then
270             write (6,*) task
271             write (6,*) 'Final X='
272             write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
273          endif
274
275 c          go back to the minimization routine.
276          goto 111
277
278       endif
279
280 c           ---------- the end of the loop -------------
281
282 c     If task is neither FG nor NEW_X we terminate execution.
283
284       if (json_active) then
285          call json_write_aggregate(trim(lbfgsb_json), task, f,
286      +                             dsave(13), n, x)
287       endif
288
289       stop
290
291       end
292
293 c======================= The end of driver3 ============================
294
```

13.6 driver3.f90

This time-controlled driver shows that it is possible to terminate a run by elapsed CPU time, and yet be able to print all desired information. This driver also illustrates the use of two stopping criteria that may be used in conjunction with a limit on execution time. The sample problem used here is the same as in driver1 and driver2 (the extended Rosenbrock function with bounds on the variables). (Fortran-90 version)

```
1 !> \file driver3.f90
2
3 !
4 !  L-BFGS-B is released under the "New BSD License" (aka "Modified BSD License"
5 !  or "3-clause license")
6 !  Please read attached file License.txt
7 !
8 !                           DRIVER 3  in Fortran 90
9 !     ----------------------------------------------------------------
10 !           TIME-CONTROLLED DRIVER FOR L-BFGS-B
11 !     ----------------------------------------------------------------
12 !
13 !        L-BFGS-B is a code for solving large nonlinear optimization
14 !             problems with simple bounds on the variables.
15 !
16 !        The code can also be used for unconstrained problems and is
17 !        as efficient for these problems as the earlier limited memory
18 !                         code L-BFGS.
19 !
20 !        This driver shows how to terminate a run after some prescribed
21 !        CPU time has elapsed, and how to print the desired information
22 !        before exiting.
23 !
24 !     References:
25 !
26 !        [1] R. H. Byrd, P. Lu, J. Nocedal and C. Zhu, "A limited
27 !        memory algorithm for bound constrained optimization",
28 !        SIAM J. Scientific Computing 16 (1995), no. 5, pp. 1190--1208.
29 !
30 !        [2] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, "L-BFGS-B: FORTRAN
31 !        Subroutines for Large Scale Bound Constrained Optimization"
32 !        Tech. Report, NAM-11, EECS Department, Northwestern University,
```

```
!        1994.
!
!
!          (Postscript files of these papers are available via anonymous
!           ftp to eecs.nwu.edu in the directory pub/lbfgs/lbfgs_bcm.)
!
!                              *  *  *
!
!         February 2011   (latest revision)
!         Optimization Center at Northwestern University
!         Instituto Tecnologico Autonomo de Mexico
!
!         Jorge Nocedal and Jose Luis Morales
!
!     **************

      program driver
      use json_writer, only: json_write_aggregate

!     This time-controlled driver shows that it is possible to terminate
!     a run by elapsed CPU time, and yet be able to print all desired
!     information. This driver also illustrates the use of two
!     stopping criteria that may be used in conjunction with a limit
!     on execution time. The sample problem used here is the same as in
!     driver1 and driver2 (the extended Rosenbrock function with bounds
!     on the variables).

      implicit none

!     We specify a limit on the CPU time (tlimit = 10 seconds)
!
!     We suppress the default output (iprint = -1). The user could
!       also elect to use the default output by choosing iprint >= 0.)
!     We suppress the code-supplied stopping tests because we will
!       provide our own termination conditions
!     We specify the dimension n of the sample problem and the number
!        m of limited memory corrections stored.

      integer,  parameter    :: n = 1000, m = 10, iprint = -1
      integer,  parameter    :: dp = kind(1.0d0)
      real(dp), parameter    :: factr  = 0.0d0, pgtol  = 0.0d0
      real(dp)               :: tlimit
!
      character(len=60)      :: task, csave
      logical                :: lsave(4)
      integer                :: isave(44)
      real(dp)               :: f
      real(dp)               :: dsave(29)
      integer,  allocatable  :: nbd(:), iwa(:)
      real(dp), allocatable  :: x(:), l(:), u(:), g(:), wa(:)
!
      real(dp)               :: t1, t2, time1, time2
      integer                :: i, j

!     Test instrumentation: dump per-iteration state to JSON when the
!     environment variable LBFGSB_JSON_OUTPUT is set. No-op otherwise.
!     LBFGSB_TLIMIT optionally overrides tlimit for reproducibility.
      character(len=512)     :: lbfgsb_json
      character(len=64)      :: env_tlimit
      logical                :: json_active

      allocate ( nbd(n), x(n), l(n), u(n), g(n) )
      allocate ( iwa(3*n) )
      allocate ( wa(2*m*n + 5*n + 11*m*m + 8*m) )

!     This time-controlled driver shows that it is possible to terminate
!     a run by elapsed CPU time, and yet be able to print all desired
!      information. This driver also illustrates the use of two
!      stopping criteria that may be used in conjunction with a limit
!      on execution time. The sample problem used here is the same as in
!      driver1 and driver2 (the extended Rosenbrock function with bounds
!      on the variables).

!     We now specify nbd which defines the bounds on the variables:
!                     l   specifies the lower bounds,
!                     u   specifies the upper bounds.

!     First set bounds on the odd-numbered variables.

      do 10 i=1, n,2
         nbd(i)=2
         l(i)=1.0d0
         u(i)=1.0d2
   10  continue

!     Next set bounds on the even-numbered variables.

```

```
120       do 12 i=2, n,2
121          nbd(i)=2
122          l(i)=-1.0d2
123          u(i)=1.0d2
124    12   continue
125
126 !     We now define the starting point.
127
128       do 14 i=1, n
129          x(i)=3.0d0
130    14 continue
131
132 !     We now write the heading of the output.
133
134       write (6,16)
135    16 format(/,5x, 'Solving sample problem.',&
136              /,5x, ' (f = 0.0 at the optimal solution.)',/)
137
138 !     We start the iteration by initializing task.
139
140       task = 'START'
141
142 !     Initialise tlimit (with optional override from LBFGSB_TLIMIT) and
143 !     enable JSON output if LBFGSB_JSON_OUTPUT is set.
144       tlimit = 10.0d0
145       call get_environment_variable('LBFGSB_TLIMIT', env_tlimit)
146       if (len_trim(env_tlimit) > 0) read(env_tlimit, *) tlimit
147       call get_environment_variable('LBFGSB_JSON_OUTPUT', lbfgsb_json)
148       json_active = (len_trim(lbfgsb_json) > 0)
149
150 !        ------- the beginning of the loop ----------
151
152 !     We begin counting the CPU time.
153
154       call timer(time1)
155
156       do while( task(1:2).eq.'FG'.or.task.eq.'NEW_X'.or. &
157                 task.eq.'START')
158
159 !     This is the call to the L-BFGS-B code.
160
161          call setulb(n,m,x,l,u,nbd,f,g,factr,pgtol,wa,iwa, &
162                      task,iprint, csave,lsave,isave,dsave)
163
164          if (task(1:2) .eq. 'FG') then
165
166 !        the minimization routine has returned to request the
167 !        function f and gradient g values at the current x.
168 !        Before evaluating f and g we check the CPU time spent.
169
170          call timer(time2)
171          if (time2-time1 .gt. tlimit) then
172             task='STOP: CPU EXCEEDING THE TIME LIMIT.'
173
174 !          Note: Assigning task(1:4)='STOP' will terminate the run;
175 !          setting task(7:9)='CPU' will restore the information at
176 !          the latest iterate generated by the code so that it can
177 !          be correctly printed by the driver.
178
179 !          In this driver we have chosen to disable the
180 !          printing options of the code (we set iprint=-1);
181 !          instead we are using customized output: we print the
182 !          latest value of x, the corresponding function value f and
183 !          the norm of the projected gradient |proj g|.
184
185 !          We print out the information contained in task.
186
187             write (6,*) task
188
189 !          We print the latest iterate contained in wa(j+1:j+n), where
190
191             j = 3*n+2*m*n+11*m**2
192             write (6,*) 'Latest iterate X ='
193             write (6,'((1x,1p, 6(1x,d11.4)))') (wa(i),i = j+1,j+n)
194
195 !          We print the function value f and the norm of the projected
196 !          gradient |proj g| at the last iterate; they are stored in
197 !          dsave(2) and dsave(13) respectively.
198
199             write (6,'(a,1p,d12.5,4x,a,1p,d12.5)') &
200             'At latest iterate   f =',dsave(2),'|proj g| =',dsave(13)
201           else
202
203 !          The time limit has not been reached and we compute
204 !          the function value f for the sample problem.
205
206             f=.25d0*(x(1)-1.d0)**2
```

```
207               do 20 i=2, n
208                  f=f+(x(i)-x(i-1)**2)**2
209    20         continue
210               f=4.d0*f
211
212  !          Compute gradient g for the sample problem.
213
214                t1 = x(2) - x(1)**2
215                g(1) = 2.d0*(x(1)-1.d0)-1.6d1*x(1)*t1
216                do 22 i=2,n-1
217                   t2=t1
218                   t1=x(i+1)-x(i)**2
219                   g(i)=8.d0*t2-1.6d1*x(i)*t1
220    22          continue
221                g(n)=8.d0*t1
222             endif
223
224  !          go back to the minimization routine.
225          else
226
227            if (task(1:5) .eq. 'NEW_X') then
228
229  !         the minimization routine has returned with a new iterate.
230  !         The time limit has not been reached, and we test whether
231  !         the following two stopping tests are satisfied:
232
233  !         1) Terminate if the total number of f and g evaluations
234  !              exceeds 900.
235
236               if (isave(34) .ge. 900) &
237               task='STOP: TOTAL NO. of f AND g EVALUATIONS EXCEEDS LIMIT'
238
239  !         2) Terminate if  |proj g|/(1+|f|) < 1.0d-10.
240
241               if (dsave(13) .le. 1.d-10*(1.0d0 + abs(f))) &
242               task='STOP: THE PROJECTED GRADIENT IS SUFFICIENTLY SMALL'
243
244  !         We wish to print the following information at each iteration:
245  !           1) the current iteration number, isave(30),
246  !           2) the total number of f and g evaluations, isave(34),
247  !           3) the value of the objective function f,
248  !           4) the norm of the projected gradient,  dsve(13)
249  !
250  !         See the comments at the end of driver1 for a description
251  !         of the variables isave and dsave.
252
253               write (6,'(2(a,i5,4x),a,1p,d12.5,4x,a,1p,d12.5)') 'Iterate' &
254               ,isave(30),'nfg =',isave(34),'f =',f,'|proj g| =',dsave(13)
255
256  !         If the run is to be terminated, we print also the information
257  !         contained in task as well as the final value of x.
258
259               if (task(1:4) .eq. 'STOP') then
260                  write (6,*) task
261                  write (6,*) 'Final X='
262                  write (6,'((1x,1p, 6(1x,d11.4)))') (x(i),i = 1,n)
263               endif
264
265             endif
266           end if
267        end do
268
269  !     If task is neither FG nor NEW_X we terminate execution.
270
271        if (json_active) &
272           call json_write_aggregate(trim(lbfgsb_json), task, f, dsave(13), n, x)
273
274        end program driver
275
276  !======================= The end of driver3 ============================
277
```

Index